

Full Stack Interview Bank

The questions that come up most often in full stack interviews, gathered from published 2026 interview guides and grouped by topic. Each one has a short model answer. Search, filter by level, mark what you know, or run a random practice round.

241 questions 0 marked known 241 shown

HTML

Semantics, accessibility, forms, performance hints. Usually the warm-up round.

What is semantic HTML and why does it matter?

JUNIOR

Using elements that describe meaning (`<header>` , `<nav>` , `<main>` , `<article>` , `<button>`) instead of generic `<div>` s.

- Screen readers use it to navigate by landmarks and headings.
- Search engines understand page structure better.
- Native elements bring keyboard and focus behaviour for free.

Difference between block, inline and inline-block elements?

JUNIOR

- **Block** (`div` , `p`): starts on a new line, takes full width, respects width/height.
- **Inline** (`span` , `a`): flows in text, ignores width/height and vertical margins.
- **Inline-block**: flows inline but respects width, height and padding.

What do `async` and `defer` do on a script tag?

JUNIOR

- No attribute: HTML parsing stops while the script downloads and runs.
- `async` : downloads in parallel, runs as soon as it arrives; order not guaranteed. Good for analytics.
- `defer` : downloads in parallel, runs after parsing, in document order. Good for app scripts.

What is the purpose of the `<!DOCTYPE html>` and the viewport meta tag?

JUNIOR

DOCTYPE puts the browser in standards mode instead of quirks mode. `<meta name="viewport" content="width=device-width, initial-scale=1">` tells mobile browsers to use the device width, so responsive CSS works.

How do you make a form accessible?

MID

- Every input has a `<label for>` (placeholder is not a label).
- Group related controls with `<fieldset>` and `<legend>`.
- Use correct `type` (email, tel) and `autocomplete` (e.g. `current-password`, `new-password`).
- Link errors with `aria-describedby` and announce with `aria-live`.
- Everything reachable and operable by keyboard.

When should you use ARIA?

MID

Only when no native element does the job. First rule of ARIA: prefer a native `<button>` over `<div role="button">`. Use ARIA for custom widgets (tabs, comboboxes), live regions and states like `aria-expanded`. Wrong ARIA is worse than none.

localStorage vs sessionStorage vs cookies?

MID

- **localStorage**: ~5 MB, persists until cleared, JS-readable, never sent to server.
- **sessionStorage**: same, but scoped to the tab and cleared when it closes.
- **Cookies**: ~4 KB, sent with every request, can be `HttpOnly`, `Secure`, `SameSite`. Use for session/refresh tokens.

Follow-up: never store auth tokens in localStorage; any XSS can read them.

What are preload, prefetch and preconnect?

MID

- **preload**: fetch a resource needed for this page now (hero image, font).
- **prefetch**: low-priority fetch for a likely next navigation.
- **preconnect**: open DNS/TCP/TLS to an origin early.

How would you improve Core Web Vitals (LCP, INP, CLS)?

SENIOR

- **LCP**: preload hero image, serve from CDN, SSR/SSG the above-the-fold content, compress images (AVIF/WebP).
- **INP**: break long tasks, reduce JS, defer non-critical scripts, use `startTransition`.
- **CLS**: set width/height or `aspect-ratio` on media, reserve space for ads/embeds, use `font-display` with matched fallbacks.

CSS

Layout (flex, grid), cascade and specificity, responsive design, modern features.

Explain the CSS box model.

JUNIOR

Every element is content + padding + border + margin. With `box-sizing: content-box` (default) width covers content only; with `border-box` width includes padding and border, which is why most resets set `*{box-sizing:border-box}`.

How is specificity calculated?

JUNIOR

Compared as a tuple (inline, IDs, classes/attributes/pseudo-classes, elements). `#nav .item a` = (0,1,1,1). Higher wins; ties go to the later rule. `!important` overrides all normal declarations. `:where()` has zero specificity; `:is()` takes its most specific argument.

Flexbox vs Grid: when to use which?

JUNIOR

Flexbox is one-dimensional: a row or a column of items, content decides size (nav bars, button groups). Grid is two-dimensional: you define rows and columns and place items (page layouts, card galleries). They combine well: grid for the page, flex inside components.

How do you center a div horizontally and vertically?

JUNIOR

```
.parent { display: grid; place-items: center; }  
/* or */  
.parent { display: flex; justify-content: center; align-items: center; }
```

Explain `position`: static, relative, absolute, fixed, sticky.

MID

- **relative**: offset from its normal spot, still takes space; becomes containing block for absolute children.
- **absolute**: removed from flow, positioned against nearest positioned ancestor.
- **fixed**: positioned against the viewport.
- **sticky**: normal until a scroll threshold, then sticks within its parent.

What creates a stacking context, and why doesn't my z-index work?

MID

A stacking context is created by `position + z-index`, `opacity < 1`, `transform`, `filter`, `isolation: isolate` and more. z-index only compares siblings inside the same context, so a child with z-index 9999 cannot escape a parent context that sits below another element.

How do you build a responsive layout? Mobile-first?

MID

Mobile-first: write base styles for small screens, add `@media (min-width: ...)` for larger. Use fluid units (`%`, `rem`, `clamp()`), `grid-template-columns: repeat(auto-fit, minmax(240px, 1fr))`, and container queries for component-level responsiveness.

What are CSS custom properties and how do they differ from Sass variables?

MID

Custom properties (`--brand: #2c56c9`) live in the browser, cascade, inherit and can change at runtime (theming, dark mode via `data-theme`). Sass variables are compiled away at build time and cannot change after.

rem vs em vs px vs vh?

MID

`px` is absolute. `em` is relative to the element's font size (compounds when nested). `rem` is relative to the root font size (predictable, respects user settings). `vh/vw` are viewport percentages; prefer `svh` on mobile to avoid the address-bar jump.

What are container queries and `:has()` ?

SENIOR

`@container (min-width: 400px)` styles a component by its container's size, not the viewport, so the same card works in a sidebar and a main column. `:has()` is a parent selector: `form:has(input:invalid) button { opacity:.5 }`.

CSS Modules vs Tailwind vs CSS-in-JS in a Next.js app?

SENIOR

- **CSS Modules:** scoped class names, zero runtime, works with Server Components.
- **Tailwind:** utility classes, zero runtime, small final CSS, fast iteration; can make markup noisy.
- **Runtime CSS-in-JS** (styled-components, Emotion): needs client components and adds runtime cost; not a good fit for RSC. Zero-runtime options (vanilla-extract, Panda) avoid that.

How do you avoid layout thrashing and janky animations?

SENIOR

Animate only `transform` and `opacity` (compositor-only). Batch DOM reads before writes; don't read `offsetHeight` inside a loop that writes styles. Use `will-change` sparingly and respect `prefers-reduced-motion`.

JavaScript & TypeScript

The language fundamentals every React and Next.js question sits on.

var vs let vs const ?

JUNIOR

`var` is function-scoped and hoisted as `undefined`. `let` / `const` are block-scoped and in the temporal dead zone until declared. `const` prevents reassignment, not mutation of the object.

== vs === ?

JUNIOR

`==` coerces types (`'1' == 1` is true, `null == undefined` is true). `===` compares type and value. Use `===`.

What is a closure? Give a use.

JUNIOR

A function that remembers variables from the scope where it was created, even after that scope finished.

```
function counter() {  
  let n = 0;  
  return () => ++n;  
}  
const next = counter(); next(); // 1, next(); // 2
```

Used for data privacy, memoization, debounce, and React hooks themselves.

Explain the event loop, microtasks and macrotasks.

MID

JS runs one call stack. When it empties, the loop drains **all microtasks** (Promise callbacks, `queueMicrotask`) then takes **one macrotask** (`setTimeout`, I/O, events), then renders if needed.

```
console.log(1);  
setTimeout(() => console.log(2));  
Promise.resolve().then(() => console.log(3));  
console.log(4); // 1 4 3 2
```

How does this work? Arrow functions?

MID

`this` depends on how a function is called: method call → the object; plain call → `undefined` in strict mode; `new` → the new instance; `call`/`apply`/`bind` → explicit. Arrow functions have no own `this`; they take it from the enclosing scope.

Promise.all vs allSettled vs race vs any?

MID

- `all` : resolves when all resolve, rejects on first rejection.
- `allSettled` : waits for all, gives status of each.
- `race` : first to settle (resolve or reject).
- `any` : first to resolve; rejects only if all reject.

Implement debounce.

MID

```
function debounce(fn, ms) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), ms);
  };
}
```

Debounce waits for a pause (search input). Throttle runs at most once per interval (scroll).

What is event delegation?

MID

Attach one listener on a parent and use `event.target.closest(...)` to handle clicks on many children, including ones added later. Works because events bubble.

Shallow vs deep copy?

MID

Spread and `Object.assign` copy one level; nested objects are shared. `structuredClone(obj)` does a real deep copy (handles Dates, Maps, cycles; not functions).

TypeScript: `interface` vs `type`, and what are generics?

SENIOR

Both describe shapes. Interfaces can be merged and extended; types can express unions, intersections, mapped and conditional types. Generics make reusable typed code:

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
```

Also know `unknown` vs `any`, `Partial`, `Pick`, `Omit`, `Record`, and discriminated unions.

How does prototypal inheritance work?

SENIOR

Each object has a hidden `[[Prototype]]` link. Property lookup walks the chain until found or `null`. `class` is syntax sugar over this: methods live on `Class.prototype`.

JS Coding & Output

Machine-round questions: predict the output, write polyfills and utilities by hand.

Predict the output: closures in a loop with `var` vs `let` .

MID

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i));
// 3 3 3 (one shared i, loop finished before callbacks run)
for (let j = 0; j < 3; j++) setTimeout(() => console.log(j));
// 0 1 2 (let creates a new binding per iteration)
```

Predict the output: event loop ordering.

MID

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'))
    .then(() => console.log('D'));
queueMicrotask(() => console.log('E'));
(async () => { console.log('F'); await null; console.log('G'); })();
console.log('H');
// A F H C E G D B
```

Sync first (A, F, H), then all microtasks in queue order (C, E, G, then D which was queued by C), then the macrotask (B).

Predict the output: `this` in methods and arrows.

MID

```
const obj = {
  name: 'x',
  regular() { return this.name; },
  arrow: () => this?.name,
  nested() { return [1].map(() => this.name); }
};
obj.regular();           // 'x'
obj.arrow();             // undefined (this from module scope)
obj.nested();            // ['x'] (arrow takes nested's this)
const f = obj.regular; f(); // TypeError in strict mode (this is undefined)
```

Write a polyfill for `Array.prototype.map`, `filter` and `reduce`.

MID

```
Array.prototype.myMap = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++)
    if (i in this) out[i] = cb.call(thisArg, this[i], i, this);
  return out;
};
Array.prototype.myFilter = function (cb) {
  const out = [];
  for (let i = 0; i < this.length; i++)
    if (i in this && cb(this[i], i, this)) out.push(this[i]);
  return out;
};
Array.prototype.myReduce = function (cb, init) {
  let i = 0, acc = init;
  if (arguments.length < 2) {
    if (!this.length) throw new TypeError('Reduce of empty array with no initial value');
    acc = this[0]; i = 1;
  }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this);
  return acc;
};
```

Write a polyfill for `Promise.all`.

SENIOR

```
function promiseAll(list) {
  return new Promise((resolve, reject) => {
    const results = [];
    let done = 0;
    if (list.length === 0) return resolve([]);
    list.forEach((p, i) => {
      Promise.resolve(p).then(v => {
        results[i] = v;           // keep original order
        if (++done === list.length) resolve(results);
      }, reject);                // first rejection wins
    });
  });
}
```

Write `Function.prototype.bind` and a throttle function.

MID

```
Function.prototype.myBind = function (ctx, ...pre) {
  const fn = this;
  return function (...args) { return fn.apply(ctx, [...pre, ...args]); };
};

function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
}
```

Implement a deep clone and a deep equal.

SENIOR

```
function deepClone(v, seen = new WeakMap()) {
  if (v === null || typeof v !== 'object') return v;
  if (v instanceof Date) return new Date(v);
  if (seen.has(v)) return seen.get(v); // handle cycles
  const out = Array.isArray(v) ? [] : {};
  seen.set(v, out);
  for (const k of Object.keys(v)) out[k] = deepClone(v[k], seen);
  return out;
}

function deepEqual(a, b) {
  if (a === b) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every(k => deepEqual(a[k], b[k]));
}
```

Flatten a nested array without `.flat()`.

MID

```
function flatten(arr, depth = Infinity) {
  return arr.reduce((acc, x) =>
    Array.isArray(x) && depth > 0
      ? acc.concat(flatten(x, depth - 1))
      : acc.concat([x]), []);
}

flatten([1, [2, [3, [4]]]]); // [1,2,3,4]
flatten([1, [2, [3, [4]]]], 1); // [1,2,[3,[4]]]
```

Implement an event emitter (pub/sub).

SENIOR

```
class Emitter {
  #map = new Map();
  on(evt, fn) {
    if (!this.#map.has(evt)) this.#map.set(evt, new Set());
    this.#map.get(evt).add(fn);
    return () => this.off(evt, fn); // unsubscribe
  }
  off(evt, fn) { this.#map.get(evt)?.delete(fn); }
  once(evt, fn) {
    const off = this.on(evt, (...a) => { off(); fn(...a); });
  }
  emit(evt, ...args) { this.#map.get(evt)?.forEach(fn => fn(...args)); }
}
```

Write a fetch wrapper with retry, timeout and exponential backoff.

SENIOR

```
async function fetchRetry(url, opts = {}, retries = 3, timeout = 5000) {
  for (let attempt = 0; ; attempt++) {
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), timeout);
    try {
      const res = await fetch(url, { ...opts, signal: ctrl.signal });
      if (res.status >= 500 && attempt < retries) throw new Error('retryable');
      return res;
    } catch (err) {
      if (attempt >= retries) throw err;
      await new Promise(r => setTimeout(r, 2 ** attempt * 300 + Math.random() * 100));
    } finally { clearTimeout(timer); }
  }
}
```

Only retry idempotent requests (GET) or ones with an idempotency key.

Curry a function: `sum(1)(2)(3)` .

MID

```
function curry(fn) {
  return function c(...args) {
    return args.length >= fn.length
      ? fn.apply(this, args)
      : (...more) => c.apply(this, [...args, ...more]);
  };
}

const add3 = curry((a, b, c) => a + b + c);
add3(1)(2)(3); add3(1, 2)(3); // 6
```

Memoize a function.

MID

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
}
```

React

Rendering model, hooks, state, performance, patterns.

What is the Virtual DOM and reconciliation?

JUNIOR

React builds a tree of elements in memory, diffs it against the previous one on each render, and applies only the minimal DOM changes. Elements of a different type are replaced; lists are matched by `key`.

Why do list items need a `key`? Why not use the index?

JUNIOR

Keys let React match items between renders. With index keys, inserting or reordering shifts keys, so React reuses the wrong component state (inputs show wrong values). Use a stable id.

Props vs state?

JUNIOR

Props are inputs passed from the parent, read-only. State is data a component owns and changes with a setter, which triggers a re-render.

Controlled vs uncontrolled components?

JUNIOR

Controlled: the input value lives in React state (`value` + `onChange`). Uncontrolled: the DOM holds the value and you read it via a ref or `FormData`. Controlled gives instant validation; uncontrolled is simpler and works well with Server Actions.

Explain `useEffect` : dependency array and cleanup.

MID

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup
}, [tick]);
```

- No array: runs after every render. `[]` : once after mount. `[a,b]` : when those change.
- Cleanup runs before the next effect and on unmount.
- Don't use effects to derive state; compute it during render instead.

`useMemo` vs `useCallback` vs `React.memo` ?

MID

`useMemo` caches a computed value; `useCallback` caches a function reference; `React.memo` skips re-rendering a component when props are shallow-equal. They work together: memo'd child + stable callback. With the React Compiler (React 19), much of this is done automatically.

When does a component re-render?

MID

When its state changes, its parent re-renders, or a context it reads changes. Props changing is not itself the trigger; the parent re-rendering is.

`useRef` uses?

MID

Access a DOM node (focus an input) and hold a mutable value that survives renders without causing a re-render (timer ids, previous value).

Context API vs Redux / Zustand? How do you pick state management?

MID

- Local UI state → `useState` .
- Server data → TanStack Query or RSC fetches (caching, refetch, dedupe).
- Rarely changing global values (theme, user) → Context.
- Frequent global client state → Zustand or Redux Toolkit (selectors avoid re-rendering every consumer).
- Shareable UI state → URL search params.

What is a custom hook? Write one.

MID

```
function useDebounce(value, ms = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setV(value), ms);
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}
```

A function starting with `use` that calls other hooks to share stateful logic.

Rules of hooks?

MID

Call hooks only at the top level (not in loops, conditions, or after early returns) and only from components or custom hooks. React relies on call order to match state to hooks.

What is new in React 19?

SENIOR

- Actions: async functions in transitions, with `useActionState` and `useFormStatus`.
- `useOptimistic` for optimistic UI.
- `use()` to read promises and context during render.
- Server Components and Server Actions stable.
- `ref` as a normal prop (no `forwardRef`), document metadata in components.
- React Compiler for automatic memoization.

Explain Suspense, `useTransition` and concurrent rendering.

SENIOR

Concurrent rendering lets React pause and prioritise work. `startTransition` marks an update as non-urgent so typing stays responsive while a heavy list re-renders. Suspense shows a fallback while children wait for code or data, and enables streaming HTML on the server.

How do you find and fix performance problems in a React app?

SENIOR

1. Measure: React DevTools Profiler, Lighthouse, Web Vitals.
2. Stop unnecessary renders: move state down, split context, memoize.
3. Virtualize long lists (TanStack Virtual).
4. Code-split with `lazy()` / dynamic imports.
5. Move data fetching and heavy libraries to Server Components.

What are Error Boundaries and what don't they catch?

SENIOR

Class components with `getDerivedStateFromError` / `componentDidCatch` that catch render errors in their subtree and show a fallback. They don't catch errors in event handlers, async code, or the boundary itself. In Next.js, `error.tsx` is an error boundary per route segment.

React & Next.js Coding

Build-it-live tasks: hooks, forms, lists, modals, auth middleware.

Write a `useFetch` hook that handles loading, errors and race conditions.

MID

```
function useFetch(url) {
  const [state, setState] = useState({ data: null, error: null, loading: true });
  useEffect(() => {
    const ctrl = new AbortController();
    setState(s => ({ ...s, loading: true }));
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(data => setState({ data, error: null, loading: false }))
      .catch(err => {
        if (err.name !== 'AbortError') setState({ data: null, error: err, loading: false });
      });
    return () => ctrl.abort(); // cancels stale request when url changes
  }, [url]);
  return state;
}
```

Follow-up: in real apps prefer TanStack Query (caching, retries, dedupe) or Server Components.

Build a debounced search box that calls an API.

MID

```
function Search() {
  const [q, setQ] = useState('');
  const debounced = useDebounce(q, 300);
  const [results, setResults] = useState([]);

  useEffect(() => {
    if (!debounced) return setResults([]);
    const ctrl = new AbortController();
    fetch('/api/search?q=' + encodeURIComponent(debounced), { signal: ctrl.signal })
      .then(r => r.json()).then(setResults).catch(() => {});
    return () => ctrl.abort();
  }, [debounced]);

  return (<>
    <input value={q} onChange={e => setQ(e.target.value)} aria-label="Search" />
    <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
  </>);
}
```

Build a Todo list with add, toggle, delete and filter.

MID

```
function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');
  const [filter, setFilter] = useState('all');

  const add = e => {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos(t => [...t, { id: crypto.randomUUID(), text, done: false }]);
    setText('');
  };

  const toggle = id => setTodos(t => t.map(x => x.id === id ? { ...x, done: !x.done } : x));
  const remove = id => setTodos(t => t.filter(x => x.id !== id));
  const visible = todos.filter(t =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done);

  return (<form onSubmit={add}>
    <input value={text} onChange={e => setText(e.target.value)} />
    <button>Add</button>
    {[ 'all', 'active', 'done' ].map(f =>
      <button type="button" key={f} onClick={() => setFilter(f)}>{f}</button>)}
    <ul>{visible.map(t => <li key={t.id}>
      <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
      {t.text} <button type="button" onClick={() => remove(t.id)}>x</button>
    </li>)}</ul>
    </form>);
}
```

Interviewers watch for immutable updates, stable keys and derived (not duplicated) state.

Build a paginated / infinite-scroll list.

MID

```
function Feed() {
  const [items, setItems] = useState([]);
  const [cursor, setCursor] = useState(null);
  const [hasMore, setHasMore] = useState(true);
  const sentinel = useRef(null);

  useEffect(() => {
    const io = new IntersectionObserver(async ([entry]) => {
      if (!entry.isIntersecting || !hasMore) return;
      const res = await fetch('/api/posts?cursor=' + (cursor ?? ''));
      const { data, next } = await res.json();
      setItems(x => [...x, ...data]); setCursor(next); setHasMore(!!next);
    });
    if (sentinel.current) io.observe(sentinel.current);
    return () => io.disconnect();
  }, [cursor, hasMore]);

  return (<>{items.map(p => <Post key={p.id} {...p} />)}<div ref={sentinel} /></>);
}
```

Write a protected login form in Next.js with a Server Action and `useActionState` .

SENIOR

```
// app/login/actions.ts
'use server';
import { z } from 'zod';
const Schema = z.object({ email: z.string().email(), password: z.string().min(8) });

export async function login(prev, formData) {
  const parsed = Schema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: 'Enter a valid email and password.' };
  const res = await fetch(process.env.API_URL + '/auth/login', {
    method: 'POST', headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(parsed.data),
  });
  if (!res.ok) return { error: 'Invalid email or password.' };
  const { refresh_token } = await res.json();
  (await cookies()).set('rt', refresh_token, { httpOnly: true, secure: true, sameSite: 'lax',
  path: '/' });
  redirect('/dashboard');
}

// app/login/page.tsx
'use client';
export default function LoginPage() {
  const [state, action, pending] = useActionState(login, null);
  return (
    <form action={action}>
      <input name="email" type="email" autoComplete="email" required />
      <input name="password" type="password" autoComplete="current-password" required />
      {state?.error && <p role="alert">{state.error}</p>}
      <button disabled={pending}>{pending ? 'Signing in...' : 'Sign in'}</button>
    </form>);
}
```

Write Next.js middleware that protects `/dashboard` routes.

SENIOR

```
// middleware.ts
import { NextResponse } from 'next/server';
import { jwtVerify } from 'jose';

const key = new TextEncoder().encode(process.env.JWT_SECRET);

export async function middleware(req) {
  const token = req.cookies.get('session')?.value;
  try {
    if (!token) throw new Error('no token');
    await jwtVerify(token, key, { issuer: 'myapp' });
    return NextResponse.next();
  } catch {
    const url = new URL('/login', req.url);
    url.searchParams.set('next', req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
}

export const config = { matcher: ['/dashboard/:path*'] };
```

Follow-up: middleware is a first gate only; verify the session again where data is read.

Implement a simple global store with `useSyncExternalStore` .

SENIOR

```
function createStore(initial) {
  let state = initial;
  const subs = new Set();
  return {
    get: () => state,
    set: fn => { state = fn(state); subs.forEach(s => s()); },
    subscribe: cb => { subs.add(cb); return () => subs.delete(cb); },
  };
}

const cart = createStore({ items: [] });

function useStore(store, selector) {
  return useSyncExternalStore(store.subscribe, () => selector(store.get()));
}

// const count = useStore(cart, s => s.items.length);
```

This is the core idea behind Zustand.

Build an accessible modal component.

MID

```
function Modal({ open, onClose, title, children }) {
  const ref = useRef(null);
  useEffect(() => {
    const d = ref.current;
    if (open && !d.open) d.showModal(); // native focus trap + Esc
    if (!open && d.open) d.close();
  }, [open]);
  return createPortal(
    <dialog ref={ref} onClose={onClose} aria-labelledby="m-title">
      <h2 id="m-title">{title}</h2>{children}
      <button onClick={onClose}>Close</button>
    </dialog>, document.body);
}
```

Next.js

App Router, Server Components, rendering strategies, caching, Server Actions, middleware.

What is Next.js and why use it over plain React?

JUNIOR

A React framework with file-based routing, server rendering (SSR/SSG/ISR), React Server Components, API routes, image/font optimization, and built-in bundling. You get SEO, faster first paint, and a backend in one project.

App Router vs Pages Router?

JUNIOR

- **Pages** (`pages/`): every component is a client component; data via `getServerSideProps` / `getStaticProps` .
- **App** (`app/`): Server Components by default, nested layouts that don't remount, streaming, `loading.tsx` / `error.tsx` , Server Actions. The default for new projects.

What are the special files in the App Router?

JUNIOR

`page.tsx` (route UI), `layout.tsx` (shared, persists), `template.tsx` (remounts), `loading.tsx` (Suspense fallback), `error.tsx` (error boundary), `not-found.tsx` , `route.ts` (API handler), plus `middleware.ts` at the root.

Server Components vs Client Components?

MID

- **Server** (default): run only on the server, can be `async`, query the DB and read secrets directly, ship zero JS.
- **Client** (`'use client'`): needed for state, effects, event handlers and browser APIs.
- Server components can render client components and pass serializable props; client components can receive server components as `children`.

Tip: push `'use client'` as far down the tree as possible.

CSR vs SSR vs SSG vs ISR vs PPR?

MID

- **CSR**: browser fetches and renders. Dashboards behind login.
- **SSR**: HTML per request. Personalized, always fresh.
- **SSG**: HTML at build time. Blogs, docs.
- **ISR**: static, regenerated after `revalidate` seconds or on demand.
- **PPR** (Partial Prerendering): static shell served instantly with dynamic holes streamed in.

How does data fetching and caching work in the App Router?

MID

```
// async Server Component
export default async function Page() {
  const res = await fetch('https://api.example.com/posts',
    { next: { revalidate: 60, tags: ['posts'] } });
  const posts = await res.json();
  return <PostList posts={posts} />;
}
```

Control freshness with `revalidate`, `cache: 'no-store'`, `revalidatePath()` / `revalidateTag()`. Newer versions add the `'use cache'` directive and `cacheLife`. Know that caching is opt-in in Next 15+.

What are Server Actions?

MID

Async functions marked `'use server'` that run on the server and can be called from forms or client code, replacing many API routes for mutations.

```
async function createPost(formData: FormData) {
  'use server';
  const session = await auth();          // always re-check auth
  await db.post.create({ data: { title: formData.get('title') } });
  revalidatePath('/posts');
}
```

Follow-up: treat every action as a public endpoint: validate input (Zod) and check authorization inside it.

What does middleware do, and what are its limits?

MID

`middleware.ts` runs before a request is matched: redirects, rewrites, auth checks on cookies, A/B testing, geo, headers. Keep it light (it runs on every matching request). Don't rely on it as the only auth check; verify again in the data layer.

Dynamic routes, route groups and parallel routes?

MID

- `[id]` dynamic, `[...slug]` catch-all, `[...slug]` optional catch-all.
- `(marketing)` route group: organizes files without affecting the URL; allows separate layouts.
- `@modal` parallel routes and `(.)photo` intercepting routes for modals with shareable URLs.

How do you handle SEO and metadata?

MID

Export `metadata` or `generateMetadata()` from a page/layout for title, description, Open Graph. Add `sitemap.ts`, `robots.ts`, and `opengraph-image.tsx`. Server rendering makes content crawlable.

How do `next/image` and `next/font` help?

MID

`next/image` resizes, serves modern formats, lazy-loads and requires dimensions to prevent CLS.
`next/font` self-hosts fonts at build time and uses size-adjusted fallbacks to avoid layout shift.

How do you implement authentication in a Next.js App Router app?

SENIOR

- Use Auth.js (NextAuth), Clerk, or a custom session: httpOnly, Secure, SameSite cookie.
- Middleware for fast redirects of signed-out users.
- A server-only `verifySession()` in a data access layer, called in every Server Component, Route Handler and Server Action that touches data.
- Never pass secrets or full user rows to client components.

What is a hydration error and how do you fix it?

SENIOR

The server HTML doesn't match what the client renders first. Causes: `Date.now()` /random values, reading `window` or `localStorage` during render, invalid HTML nesting, browser extensions. Fix by moving browser-only logic into `useEffect`, using `dynamic(..., { ssr:false })`, or making output deterministic.

How does streaming work in Next.js?

SENIOR

The server sends the layout and fallbacks immediately, then streams each Suspense boundary's HTML as its data resolves. Wrap slow parts in `<Suspense>` or add `loading.tsx`. Start independent fetches in parallel with `Promise.all` to avoid waterfalls.

Route Handlers vs Server Actions: when to use which?

SENIOR

Server Actions for mutations from your own UI (forms, buttons). Route Handlers (`app/api/*/route.ts`) for public APIs, webhooks (Stripe), mobile clients, and GET endpoints other systems call.

How would you deploy Next.js outside Vercel, e.g. on Azure?

SENIOR

Build with `output: 'standalone'`, containerize, and run on Azure App Service (Linux), Azure Container Apps or AKS. Put Azure Front Door/CDN in front for static assets. Static-only sites can go to Azure Static Web Apps. Configure env vars via App Settings and secrets via Key Vault references.

Python

Language internals and idioms that interviewers probe.

List vs tuple vs set vs dict?

JUNIOR

- **list**: ordered, mutable, allows duplicates.
- **tuple**: ordered, immutable, hashable if contents are.
- **set**: unordered, unique items, O(1) membership.
- **dict**: key-value, insertion-ordered (3.7+), O(1) lookup.

Mutable vs immutable types, and the mutable default argument trap?

JUNIOR

```
def add(item, bucket=[]): # same list reused every call!
    bucket.append(item); return bucket

def add(item, bucket=None): # fix
    bucket = [] if bucket is None else bucket
```

Defaults are evaluated once when the function is defined.

is vs == ?

JUNIOR

`==` compares values; `is` compares identity (same object). Use `is` only for `None`, `True`, `False`.

What are `*args` and `**kwargs` ?

JUNIOR

`*args` collects extra positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict. Also used to unpack when calling: `f(*items, **opts)`.

What is a decorator? Write one.

MID

```
import functools, time
def timed(fn):
    @functools.wraps(fn)
    def wrapper(*a, **kw):
        t = time.perf_counter()
        try: return fn(*a, **kw)
        finally: print(fn.__name__, time.perf_counter() - t)
    return wrapper
```

A function that takes a function and returns a new one. Used for auth checks, caching, logging, retries.

Generators and `yield` : why use them?

MID

Generators produce values lazily, one at a time, keeping memory flat. Ideal for streaming large files or DB rows. `(x*x for x in data)` is a generator expression.

What is the GIL? Threads vs processes vs asyncio?

MID

The Global Interpreter Lock lets only one thread run Python bytecode at a time in CPython.

- I/O-bound → `asyncio` or threads.
- CPU-bound → `multiprocessing` / `ProcessPoolExecutor`.
- Python 3.13+ has an optional free-threaded (no-GIL) build; know it exists.

Explain `async` / `await` and the event loop in Python.

MID

Coroutines pause at `await` so the event loop can run others while waiting on I/O. Run many concurrently with `asyncio.gather()` or `TaskGroup`. Never call blocking code (e.g. `requests`, `time.sleep`) inside async code; use `httpx`, `asyncio.sleep`, or `run_in_executor`.

What are context managers?

MID

Objects with `__enter__` / `__exit__` used by `with` to guarantee cleanup (files, DB transactions, locks). Write one quickly with `@contextlib.contextmanager`.

Shallow vs deep copy in Python?

MID

`copy.copy()` / `list[:]` copy the outer container; inner objects are shared. `copy.deepcopy()` copies recursively.

How does memory management and garbage collection work?

SENIOR

CPython uses reference counting (freed when count hits zero) plus a generational cycle collector for reference cycles. `__slots__` reduces per-instance memory; `weakref` avoids cycles in caches.

dataclasses vs Pydantic models?

SENIOR

dataclasses generate `__init__` / `__repr__` / `__eq__` with no validation. Pydantic validates and coerces data at runtime and serializes to JSON; use it at API boundaries (FastAPI) and for settings.

Explain MRO and `super()` with multiple inheritance.

SENIOR

Method Resolution Order uses C3 linearization (see `Cls.__mro__`). `super()` calls the next class in the MRO, not necessarily the direct parent, which makes cooperative mixins work.

Python Coding

Python coding round: predict-the-output traps, lists and dicts, OOP, iterators and generators, decorators, concurrency, file handling, sorting, testing.

Predict the output: mutable default, late-binding lambdas, list aliasing.

MID

```
def f(x, acc=[]):
    acc.append(x); return acc
f(1); print(f(2))           # [1, 2] default list is shared

fns = [lambda: i for i in range(3)]
print([fn() for fn in fns]) # [2, 2, 2] i looked up at call time
fns = [lambda i=i: i for i in range(3)] # fix -> [0, 1, 2]

grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)                 # [[1,0,0],[1,0,0],[1,0,0]] same inner list 3x
grid = [[0] * 3 for _ in range(3)] # fix
```

Predict the output: identity, floats, bools, tuples.

MID

```
a = 256; b = 256; print(a is b) # True (small ints -5..256 are cached)
a = 1000; b = 1000; a is b      # may be False: never use 'is' for numbers
print(0.1 + 0.2 == 0.3)        # False -> use math.isclose()
print(True + True)              # 2 bool is a subclass of int
t = (1, [2]); t[1].append(3)    # works: tuple holds a reference to a mutable list
t[1] += [4]                     # TypeError raised, BUT list is still extended!
print({1: 'a', True: 'b'})      # {1: 'b'} 1 == True and same hash
x = [1, 2, 3]; y = x; y += [4]; print(x) # [1, 2, 3, 4] += mutates in place
```

Predict the output: scope, `global` / `nonlocal` and closures.

MID

```
x = 10
def f():
    print(x)
    x = 5          # makes x local to the WHOLE function
f()               # UnboundLocalError

def counter():
    n = 0
    def inc():
        nonlocal n # without this: UnboundLocalError
        n += 1; return n
    return inc
```

Python resolves names with LEGB: Local, Enclosing, Global, Built-in.

Swap two variables, find the missing number in 1..n, and find duplicates.

JUNIOR

```
a, b = b, a          # tuple unpacking swap

def missing(nums, n):          # O(n), O(1)
    return n * (n + 1) // 2 - sum(nums)

def duplicates(nums):
    seen, dup = set(), set()
    for x in nums:
        (dup if x in seen else seen).add(x)
    return list(dup)

from collections import Counter
[k for k, v in Counter(nums).items() if v > 1] # one-liner
```

Sort a dict by value, sort a list of dicts by multiple keys, merge dicts.

JUNIOR

```
scores = {"a": 3, "b": 1, "c": 2}
dict(sorted(scores.items(), key=lambda kv: kv[1], reverse=True))

users = [{"name": "Ana", "age": 30}, {"name": "Bo", "age": 25}, {"name": "Cy", "age": 30}]
sorted(users, key=lambda u: (-u["age"], u["name"])) # age desc, then name asc
from operator import itemgetter
sorted(users, key=itemgetter("age"))

merged = {**d1, **d2}          # d2 wins on conflicts
merged = d1 | d2               # Python 3.9+
max(scores, key=scores.get)    # key with highest value -> 'a'
```

List/dict/set comprehensions: squares of evens, invert a dict, flatten a matrix.

JUNIOR

```
[x * x for x in nums if x % 2 == 0]
{v: k for k, v in d.items()}          # invert dict
[x for row in matrix for x in row]    # flatten 2D
{w.lower() for w in words}            # unique lowercase
{w: len(w) for w in words}
list(zip(*matrix))                    # transpose
```

Flatten an arbitrarily nested list (recursive and with a generator).

MID

```
def flatten(xs):
    out = []
    for x in xs:
        if isinstance(x, list): out.extend(flatten(x))
        else: out.append(x)
    return out

def iflatten(xs):                      # lazy version
    for x in xs:
        if isinstance(x, (list, tuple)): yield from iflatten(x)
        else: yield x

list(iflatten([1, [2, [3, (4, 5)]]])) # [1, 2, 3, 4, 5]
```

Split a list into chunks of size n; sliding windows.

MID

```
def chunks(xs, n):
    return [xs[i:i + n] for i in range(0, len(xs), n)]

from itertools import batched        # Python 3.12+
list(batched(range(7), 3))            # [(0,1,2), (3,4,5), (6,)]

def windows(xs, k):
    return [xs[i:i + k] for i in range(len(xs) - k + 1)]
```

Armstrong number, sum of digits, reverse an integer, GCD/LCM.

JUNIOR

```
def is_armstrong(n):
    d = str(n); return n == sum(int(c) ** len(d) for c in d)    # 153 -> True

def digit_sum(n): return sum(map(int, str(abs(n))))

def reverse_int(n):
    sign = -1 if n < 0 else 1
    return sign * int(str(abs(n))[::-1])

def gcd(a, b):
    while b: a, b = b, a % b
    return a

def lcm(a, b): return a * b // gcd(a, b)    # or math.gcd / math.lcm
```

Write a generator for an infinite Fibonacci sequence and take the first 10.

MID

```
from itertools import islice
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

list(islice(fib(), 10))    # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Write a custom iterator class (`__iter__` / `__next__`).

MID

```
class Countdown:
    def __init__(self, start): self.cur = start
    def __iter__(self): return self
    def __next__(self):
        if self.cur < 0: raise StopIteration
        self.cur -= 1
        return self.cur + 1

for n in Countdown(3): print(n)    # 3 2 1 0
```

An **iterable** has `__iter__` ; an **iterator** also has `__next__` and is consumed once.

Read a huge log file and count status codes without loading it into memory.

MID

```
from collections import Counter

def status_counts(path):
    counts = Counter()
    with open(path, encoding="utf-8") as f:
        for line in f:
            # file objects stream line by line
            parts = line.split()
            if len(parts) > 8: counts[parts[8]] += 1
    return counts

# top 5 IPs
def top_ips(path, k=5):
    with open(path) as f:
        return Counter(line.split(maxsplit=1)[0] for line in f).most_common(k)
```

Parse JSON/CSV and group records (e.g. total sales per region).

MID

```
import csv, json
from collections import defaultdict

def sales_by_region(path):
    totals = defaultdict(float)
    with open(path, newline="") as f:
        for row in csv.DictReader(f):
            totals[row["region"]] += float(row["amount"])
    return dict(totals)

data = json.loads('{ "users": [{ "id": 1, "tags": ["a"]} ] }')
json.dumps(data, indent=2, default=str) # default=str handles datetime
```

Validate an email and extract all numbers / URLs with regex.

MID

```
import re
EMAIL = re.compile(r"^\w.+-]+@[\w-]+(\.[\w-]+)+$")
bool(EMAIL.match("dev@example.com")) # True

re.findall(r"-?\d+(?:\.\d+)?", "a1 b-2.5 c30") # ['1', '-2.5', '30']
re.findall(r"https?:\/\/\S+", text)
re.sub(r"\s+", " ", text).strip() # collapse whitespace
```

Follow-up: real email validation is sending a verification link; regex only catches typos.

OOP: write a class with `@property`, `@classmethod`, `@staticmethod` and dunder methods.

MID

```
class Money:
    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency

    @property
    def amount(self): return self._amount
    @amount.setter
    def amount(self, v):
        if v < 0: raise ValueError("amount must be >= 0")
        self._amount = v

    @classmethod
    def from_str(cls, s):
        # alternative constructor: "100 USD"
        amt, cur = s.split(); return cls(float(amt), cur)

    @staticmethod
    def round2(x): return round(x, 2)

    def __add__(self, other):
        if self.currency != other.currency: raise ValueError("currency mismatch")
        return Money(self.amount + other.amount, self.currency)
    def __eq__(self, o): return (self.amount, self.currency) == (o.amount, o.currency)
    def __repr__(self): return f"Money({self.amount!r}, {self.currency!r})"
```

Abstract base classes and polymorphism: payment processors.

MID

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, amount: float) -> str: ...

class Stripe(PaymentGateway):
    def charge(self, amount): return f"stripe:{amount}"
class Razorpay(PaymentGateway):
    def charge(self, amount): return f"razorpay:{amount}"

def checkout(gw: PaymentGateway, amt): return gw.charge(amt)
PaymentGateway() # TypeError: can't instantiate abstract class
```

Also know `typing.Protocol` for duck-typed interfaces without inheritance.

Implement a Singleton three ways.

SENIOR

```
# 1. Module-level instance (most Pythonic)
settings = Settings()

# 2. __new__
class Singleton:
    _inst = None
    def __new__(cls, *a, **kw):
        if cls._inst is None: cls._inst = super().__new__(cls)
        return cls._inst

# 3. Metaclass
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *a, **kw):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*a, **kw)
        return cls._instances[cls]
class DB(metaclass=SingletonMeta): ...
```

In FastAPI you'd usually use `@lru_cache` on a `get_settings()` dependency instead.

Write a decorator that takes arguments, and a class-based decorator.

MID

```
import functools

def repeat(times):
    # decorator factory
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **kw):
            return [fn(*a, **kw) for _ in range(times)]
        return wrapper
    return deco

@repeat(3)
def hi(): return "hi" # ['hi', 'hi', 'hi']

class CountCalls:
    def __init__(self, fn):
        functools.update_wrapper(self, fn); self.fn, self.calls = fn, 0
    def __call__(self, *a, **kw):
        self.calls += 1; return self.fn(*a, **kw)
```

Implement your own `lru_cache` -style memoize decorator with a TTL.

MID

```
import functools, time

def ttl_cache(seconds):
    def deco(fn):
        cache = {}
        @functools.wraps(fn)
        def wrapper(*args):
            now = time.monotonic()
            if args in cache and now - cache[args][1] < seconds:
                return cache[args][0]
            val = fn(*args); cache[args] = (val, now); return val
        return wrapper
    return deco

@functools.lru_cache(maxsize=128)    # built-in: args must be hashable
def slow_square(n): return n * n
```

`map`, `filter`, `reduce`, `zip`, `enumerate`, `any` / `all` in one task.

MID

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
list(map(lambda x: x * 2, nums))           # [2, 4, 6, 8, 10]
list(filter(lambda x: x % 2, nums))        # [1, 3, 5]
reduce(lambda a, b: a * b, nums)           # 120
dict(zip(["a", "b"], [1, 2]))              # {'a': 1, 'b': 2}
for i, v in enumerate(nums, start=1): ...
any(x > 4 for x in nums), all(x > 0 for x in nums)  # True, True
```

Thread-safe counter, and why `count += 1` isn't atomic.

SENIOR

```
import threading
from concurrent.futures import ThreadPoolExecutor

class Counter:
    def __init__(self): self.n = 0; self.lock = threading.Lock()
    def inc(self):
        with self.lock:          # read-modify-write must be protected
            self.n += 1

c = Counter()
with ThreadPoolExecutor(8) as ex:
    for _ in range(10_000): ex.submit(c.inc)
print(c.n)    # 10000
```

The GIL doesn't make `+=` atomic: it's several bytecodes (load, add, store) and a thread switch can happen in between.

Producer-consumer with `asyncio.Queue`.

SENIOR

```
import asyncio

async def producer(q, n):
    for i in range(n): await q.put(i)
    for _ in range(3): await q.put(None)    # one stop signal per worker

async def worker(name, q):
    while (item := await q.get()) is not None:
        await asyncio.sleep(0.1)           # simulate I/O
        print(name, item)
        q.task_done()
    q.task_done()

async def main():
    q = asyncio.Queue(maxsize=10)           # back-pressure
    async with asyncio.TaskGroup() as tg:
        tg.create_task(producer(q, 20))
        for i in range(3): tg.create_task(worker(f"w{i}", q))

asyncio.run(main())
```

CPU-bound work: run it in parallel with processes.

MID

```
from concurrent.futures import ProcessPoolExecutor

def is_prime(n):
    if n < 2: return False
    return all(n % i for i in range(2, int(n ** 0.5) + 1))

if __name__ == "__main__":
    # required on Windows/macOS spawn
    with ProcessPoolExecutor() as ex:
        primes = [n for n, p in zip(nums, ex.map(is_prime, nums, chunksize=1000)) if p]
```

Custom exceptions and clean error handling (try/except/else/finally).

MID

```
class AppError(Exception): ...
class NotFound(AppError):
    def __init__(self, what, id): super().__init__(f"{what} {id} not found")

def load(id):
    try:
        row = db.get(id)
    except ConnectionError as e:
        raise AppError("database unavailable") from e    # keep original traceback
    else:
        if row is None: raise NotFound("user", id)
        return row
    finally:
        metrics.inc("load_attempts")
```

Catch specific exceptions, never a bare `except:`. Python 3.11 adds `ExceptionGroup` and `except*`.

Implement a Stack class with `min()` in $O(1)$.

MID

```
class MinStack:
    def __init__(self): self.s = []          # store (value, current_min)
    def push(self, x):
        m = min(x, self.s[-1][1]) if self.s else x
        self.s.append((x, m))
    def pop(self): return self.s.pop()[0]
    def top(self): return self.s[-1][0]
    def min(self): return self.s[-1][1]
```

Rotate a matrix 90° clockwise; spiral order traversal.

MID

```
def rotate(m):
    return [list(row) for row in zip(*m[::-1])]

def spiral(m):
    out = []
    while m:
        out += m.pop(0)          # top row
        m = [list(r) for r in zip(*m)][::-1] # rotate the rest counter-clockwise
    return out
spiral([[1,2,3],[4,5,6],[7,8,9]])    # [1,2,3,6,9,8,7,4,5]
```

Word frequency: top N words in a text, ignoring case and punctuation.

MID

```
import re
from collections import Counter

def top_words(text, n=3):
    words = re.findall(r"[a-z']+", text.lower())
    return Counter(words).most_common(n)

top_words("The cat and the hat. The end!")    # [('the', 3), ('cat', 1), ('and', 1)]
```

Write pytest tests with fixtures, parametrize and mocking.

SENIOR

```
import pytest
from unittest.mock import patch

@pytest.fixture
def user(): return {"id": 1, "email": "a@b.com"}

@pytest.mark.parametrize("s,expected", [("racecar", True), ("abc", False), ("", True)])
def test_palindrome(s, expected):
    assert is_palindrome(s) is expected

def test_signup_sends_email(client):
    with patch("app.auth.send_verification") as send:
        r = client.post("/auth/signup", json={"email": "a@b.com", "password": "S3cure!pass"})
        assert r.status_code == 202
        send.assert_called_once()

def test_bad_input_raises():
    with pytest.raises(ValueError): Money(-1)
```

Implement binary search, bubble sort, merge sort and quicksort in Python.

MID

```
def merge_sort(a):
    if len(a) <= 1: return a
    mid = len(a) // 2
    l, r = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] <= r[j]: out.append(l[i]); i += 1
        else: out.append(r[j]); j += 1
    return out + l[i:] + r[j:] # O(n log n), stable

def quick_sort(a):
    if len(a) <= 1: return a
    p = a[len(a) // 2]
    return quick_sort([x for x in a if x < p]) + [x for x in a if x == p] + quick_sort([x for x in a if x > p])

def bubble_sort(a):
    a = a[:]
    for i in range(len(a)):
        swapped = False
        for j in range(len(a) - 1 - i):
            if a[j] > a[j + 1]: a[j], a[j + 1] = a[j + 1], a[j]; swapped = True
        if not swapped: break
    return a # O(n^2)
```

Check balanced brackets, count substrings, and find all permutations of a string.

MID

```
from itertools import permutations, combinations
[''.join(p) for p in permutations("abc")] # 6 strings
list(combinations([1, 2, 3], 2)) # [(1,2),(1,3),(2,3)]

def perms(s):
    # by hand (backtracking)
    if len(s) <= 1: return [s]
    return [c + p for i, c in enumerate(s) for p in perms(s[:i] + s[i+1:])]

"banana".count("an") # 2 (non-overlapping)
sum(1 for i in range(len(s)) if s.startswith("ana", i)) # overlapping count
```

dataclass with defaults, ordering and immutability.

MID

```
from dataclasses import dataclass, field

@dataclass(order=True, frozen=True, slots=True)
class Task:
    priority: int
    title: str = field(compare=False)
    tags: list[str] = field(default_factory=list, compare=False)    # not tags=[]!

sorted([Task(2, "b"), Task(1, "a")])    # ordered by priority
t = Task(1, "x"); t.priority = 5        # FrozenInstanceError
```

Python Backend (FastAPI / Django)

API design and framework questions for the backend half of the role.

FastAPI vs Django vs Flask?

JUNIOR

- **FastAPI**: async, type hints + Pydantic validation, auto OpenAPI docs. Great for APIs.
- **Django**: batteries included (ORM, admin, auth, migrations). Great for full apps.
- **Flask**: minimal, pick your own pieces.

How does dependency injection work in FastAPI?

MID

```
async def get_db():
    async with SessionLocal() as db:
        yield db

async def current_user(token: str = Depends(oauth2_scheme),
                        db = Depends(get_db)) -> User: ...

@app.get("/me")
async def me(user: User = Depends(current_user)):
    return user
```

Depends wires DB sessions, auth and settings into routes, and makes them easy to override in tests.

How do you structure a production FastAPI project?

MID

Routers per domain (`api/users.py`), Pydantic schemas separate from ORM models, a service layer for business logic, SQLAlchemy + Alembic migrations, settings via `pydantic-settings`, pytest with a test DB, run with Uvicorn/Gunicorn workers behind a reverse proxy.

What is the N+1 query problem and how do you fix it in Django / SQLAlchemy?

MID

Loading a list then querying related rows once per item. Fix in Django with `select_related` (FK/one-to-one, JOIN) and `prefetch_related` (many-to-many, second query). In SQLAlchemy use `selectinload` / `joinedload`.

How do you run background jobs in Python?

MID

Short fire-and-forget: FastAPI `BackgroundTasks`. Reliable/retryable work (emails, reports): Celery, RQ or Dramatiq with Redis/RabbitMQ, or Azure Service Bus + Azure Functions. Make tasks idempotent.

Explain Django's request lifecycle and middleware.

MID

Request → WSGI/ASGI handler → middleware (in order) → URL resolver → view → template/serializer → middleware (reverse) → response. Middleware handles sessions, CSRF, auth, security headers.

How do you version and document a REST API?

SENIOR

URL versioning (`/api/v1`) or a header. Never break existing clients: add fields, don't rename. Publish OpenAPI (FastAPI generates it), and use it to generate a typed TypeScript client for the Next.js front end.

How do you test a FastAPI app?

SENIOR

pytest + `httpx.AsyncClient` / `TestClient`, override dependencies (`app.dependency_overrides`) for DB and auth, transactional fixtures that roll back, factory fixtures for data, and contract tests against the OpenAPI schema.

Backend Coding (Python / SQL)

Write the endpoint: auth, password reset, rate limiting, async, SQL queries.

Write FastAPI signup and login endpoints with Argon2 and JWT.

MID

```
from datetime import datetime, timedelta, timezone
from fastapi import APIRouter, Depends, HTTPException, Response
from argon2 import PasswordHasher
import jwt, secrets, hashlib

router = APIRouter(prefix="/auth")
ph = PasswordHasher()
SECRET, ALG = settings.jwt_secret, "HS256"

def make_access(user_id: str) -> str:
    now = datetime.now(timezone.utc)
    return jwt.encode({"sub": user_id, "iat": now, "exp": now + timedelta(minutes=15),
                      "iss": "myapp"}, SECRET, algorithm=ALG)

@router.post("/signup", status_code=202)
async def signup(body: SignupIn, db=Depends(get_db)):
    if not await db.users.exists(body.email):
        user = await db.users.create(email=body.email, password_hash=ph.hash(body.password))
        await send_verification(user)          # queued job
    return {"message": "Check your inbox to verify your email."} # same reply either way

@router.post("/login")
async def login(body: LoginIn, response: Response, db=Depends(get_db)):
    user = await db.users.by_email(body.email)
    try:
        ph.verify(user.password_hash if user else DUMMY_HASH, body.password)
    except Exception:
        raise HTTPException(401, "Invalid email or password")
    if not user or not user.email_verified:
        raise HTTPException(401, "Invalid email or password")
    refresh = secrets.token_urlsafe(32)
    await db.refresh_tokens.create(user_id=user.id,
                                   token_hash=hashlib.sha256(refresh.encode()).hexdigest(),
                                   expires_at=datetime.now(timezone.utc) + timedelta(days=14))
    response.set_cookie("rt", refresh, httponly=True, secure=True, samesite="lax",
                       max_age=14*24*3600, path="/auth")
    return {"access_token": make_access(str(user.id)), "token_type": "bearer"}
```

Verifying against a dummy hash when the user doesn't exist keeps response time equal, so attackers can't detect valid emails by timing.

Write the forgot-password and reset-password endpoints.

MID

```
RESET_TTL = timedelta(minutes=30)

@router.post("/forgot-password", status_code=202)
async def forgot(body: EmailIn, db=Depends(get_db), _=Depends(rate_limit("forgot", 5, 3600))):
    user = await db.users.by_email(body.email)
    if user:
        token = secrets.token_urlsafe(32)
        await db.reset_tokens.delete_for(user.id)
        await db.reset_tokens.create(user_id=user.id,
                                     token_hash=sha256(token), expires_at=now() + RESET_TTL)
        await queue.send("email.reset", {"to": user.email,
                                         "link": f"{settings.frontend_url}/reset?token={token}"})
    return {"message": "If an account exists, we sent a reset link."}

@router.post("/reset-password")
async def reset(body: ResetIn, db=Depends(get_db)):
    row = await db.reset_tokens.by_hash(sha256(body.token))
    if not row or row.expires_at < now():
        raise HTTPException(400, "This link is invalid or has expired.")
    async with db.transaction():
        await db.users.set_password(row.user_id, ph.hash(body.new_password))
        await db.reset_tokens.delete_for(row.user_id)
        await db.refresh_tokens.revoke_all(row.user_id)    # log out everywhere
    await queue.send("email.password_changed", {"user_id": row.user_id})
    return {"message": "Password updated. Please sign in."}
```

The link is built from a configured `frontend_url`, never from the request's Host header (host header injection).

Write a FastAPI dependency that gets the current user from a JWT and checks a role.

MID

```
from fastapi.security import HTTPBearer
bearer = HTTPBearer()

async def current_user(cred=Depends(bearer), db=Depends(get_db)) -> User:
    try:
        payload = jwt.decode(cred.credentials, SECRET, algorithms=["HS256"],
                               issuer="myapp", options={"require": ["exp", "sub"]})
    except jwt.PyJWTError:
        raise HTTPException(401, "Invalid or expired token",
                               headers={"WWW-Authenticate": "Bearer"})
    user = await db.users.get(payload["sub"])
    if not user or not user.is_active:
        raise HTTPException(401)
    return user

def require_role(*roles):
    async def check(user: User = Depends(current_user)):
        if user.role not in roles:
            raise HTTPException(403, "Not allowed")
        return user
    return check

@app.delete("/admin/users/{uid}")
async def delete_user(uid: str, admin=Depends(require_role("admin"))): ...
```

Implement a Redis sliding-window rate limiter.

SENIOR

```
import time, uuid
async def allow(redis, key: str, limit: int, window_s: int) -> bool:
    now = time.time()
    pipe = redis.pipeline()
    pipe.zremrangebyscore(key, 0, now - window_s)      # drop old hits
    pipe.zadd(key, {str(uuid.uuid4()): now})
    pipe.zcard(key)
    pipe.expire(key, window_s)
    _, _, count, _ = await pipe.execute()
    return count <= limit

def rate_limit(name, limit, window):
    async def dep(request: Request):
        key = f"rl:{name}:{request.client.host}"
        if not await allow(redis, key, limit, window):
            raise HTTPException(429, "Too many requests", headers={"Retry-After":
str(window)})
        return dep
```

Write a retry decorator with exponential backoff (sync and async).

MID

```
import asyncio, functools, random, time

def retry(times=3, base=0.3, exceptions=(Exception,)):
    def deco(fn):
        if asyncio.iscoroutinefunction(fn):
            @functools.wraps(fn)
            async def aw(*a, **kw):
                for i in range(times):
                    try: return await fn(*a, **kw)
                    except exceptions:
                        if i == times - 1: raise
                        await asyncio.sleep(base * 2**i + random.random() / 10)
                return aw
            @functools.wraps(fn)
            def w(*a, **kw):
                for i in range(times):
                    try: return fn(*a, **kw)
                    except exceptions:
                        if i == times - 1: raise
                        time.sleep(base * 2**i)
                return w
        return w
    return deco
```

Fetch many URLs concurrently with a concurrency limit.

MID

```
import asyncio, httpx

async def fetch_all(urls, limit=10):
    sem = asyncio.Semaphore(limit)
    async with httpx.AsyncClient(timeout=10) as client:
        async def one(u):
            async with sem:
                r = await client.get(u)
                return u, r.status_code
        return await asyncio.gather(*(one(u) for u in urls), return_exceptions=True)
```

Write a context manager that times a block, both class-based and with `contextlib`.

MID

```
import time
from contextlib import contextmanager

class Timer:
    def __enter__(self): self.t = time.perf_counter(); return self
    def __exit__(self, *exc): self.elapsed = time.perf_counter() - self.t

@contextmanager
def timer(label):
    t = time.perf_counter()
    try: yield
    finally: print(label, round(time.perf_counter() - t, 3))

with timer("query"): run_query()
```

Write SQL: second highest salary, top 3 per department, and users with no orders.

SENIOR

```
-- 2nd highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- top 3 earners per department
SELECT * FROM (
    SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees e
) t WHERE rnk <= 3;

-- users who never ordered
SELECT u.id, u.email FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;

-- duplicate emails
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

Upload a file directly from the browser to Azure Blob using a SAS URL (Python backend).

SENIOR

```
from azure.identity import DefaultAzureCredential
from azure.storage.blob import BlobServiceClient, generate_blob_sas, BlobSasPermissions

svc = BlobServiceClient(settings.blob_url, credential=DefaultAzureCredential())

@app.post("/uploads/sas")
async def upload_url(name: str, user=Depends(current_user)):
    blob = f"{user.id}/{uuid4()}-{secure_filename(name)}"
    start, expiry = now(), now() + timedelta(minutes=10)
    key = svc.get_user_delegation_key(start, expiry)
    sas = generate_blob_sas(svc.account_name, "uploads", blob, user_delegation_key=key,
                           permission=BlobSasPermissions(create=True, write=True),
                           expiry=expiry)
    return {"url": f"{settings.blob_url}/uploads/{blob}?{sas}"}
# Browser: PUT file to that url with header x-ms-blob-type: BlockBlob
```

The API never streams the file itself, so it stays fast; managed identity means no storage key in code.

Databases & APIs

SQL, indexing, transactions, REST vs GraphQL, caching.

SQL vs NoSQL: when to use each?

JUNIOR

SQL (PostgreSQL, Azure SQL) for relational data, joins and ACID transactions (orders, users, payments). NoSQL (Cosmos DB, MongoDB, Redis) for flexible schemas, huge scale with simple access patterns, caching, or document data. Many systems use both.

What are REST principles and common HTTP status codes?

JUNIOR

Resources as nouns (`/users/42`), HTTP verbs for actions, stateless requests. Codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthenticated, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Validation error, 429 Too Many Requests, 500 Server error.

PUT vs PATCH? Which methods are idempotent?

JUNIOR

PUT replaces the whole resource; PATCH changes some fields. GET, PUT, DELETE are idempotent (repeating gives the same result); POST is not, so use idempotency keys for payments.

What is an index and when does it hurt?

MID

A B-tree (usually) that speeds lookups, sorts and joins on the indexed columns. It costs storage and slows writes. Composite indexes follow the leftmost-prefix rule. Check with `EXPLAIN ANALYZE`.

Explain ACID and transaction isolation levels.

MID

Atomicity, Consistency, Isolation, Durability. Isolation levels trade safety for concurrency: Read Uncommitted, Read Committed (Postgres default), Repeatable Read, Serializable. Know dirty reads, non-repeatable reads, phantom reads.

Types of JOIN?

MID

INNER (matches only), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS (cartesian). Be ready to write a query like "users with no orders" using LEFT JOIN ... WHERE o.id IS NULL.

REST vs GraphQL vs tRPC?

MID

REST: simple, cacheable, standard. GraphQL: client picks fields, one endpoint, avoids over/under-fetching, harder caching and N+1 risk on the server. tRPC: end-to-end types in TypeScript monorepos only. With a Python backend, REST + OpenAPI-generated types is the usual pick.

How do you paginate an API?

MID

Offset (`?page=3`) is simple but slow on deep pages and shifts when rows are inserted. Cursor/keyset (`WHERE id > :last_id ORDER BY id LIMIT 20`) is stable and fast for infinite scroll.

Normalization vs denormalization?

SENIOR

Normalize (1NF–3NF) to remove duplication and keep writes consistent. Denormalize selectively for read-heavy paths (counters, read models, materialized views) and accept the cost of keeping copies in sync.

How do you handle database migrations with zero downtime?

SENIOR

Expand and contract: add the new column (nullable), deploy code that writes both, backfill, switch reads, then remove the old column in a later release. Avoid long table locks; create indexes `CONCURRENTLY` in Postgres.

Auth Architecture

Signup, login, email verification, password reset, sessions vs JWT, OAuth, MFA. Frequently asked as a system design round.

[See the signup, login and reset-password diagrams](#)

Authentication vs authorization?

JUNIOR

Authentication (AuthN) proves who you are (login). Authorization (AuthZ) decides what you may do (roles, permissions). A 401 means "not authenticated"; a 403 means "authenticated but not allowed".

How should passwords be stored?

JUNIOR

Never plaintext or plain SHA/MD5. Use a slow, salted, adaptive hash: **Argon2id** (preferred) or bcrypt. The salt is unique per user and stored with the hash. Compare in constant time.

```
from argon2 import PasswordHasher
ph = PasswordHasher()
hash = ph.hash(password)
ph.verify(hash, attempt) # raises on mismatch
```

Design the signup and email verification flow.

MID

1. Validate email + password strength (check against breached-password lists).
2. Create user with `email_verified=false`, hashed password.
3. Generate a random token (32 bytes, `secrets.token_urlsafe`), store **its hash** with expiry (e.g. 24h).
4. Queue email with link `/verify?token=...` (async via a job queue).
5. On click: hash the token, look it up, check expiry, mark verified, delete token.
6. Allow resend with rate limits.

Follow-up: if the email already exists, show the same message ("check your inbox") to prevent account enumeration.

Design a secure forgot / reset password flow.

MID

1. User enters email. Always respond "If an account exists, we sent a link" (no enumeration).
2. Generate a single-use, random, high-entropy token; store only its hash, expire in 15–60 minutes.
3. Email link over HTTPS. Rate-limit requests per email and IP.
4. On submit: verify token hash and expiry, set new hashed password, delete the token (and all other reset tokens).
5. **Revoke all existing sessions / refresh tokens** for that user.
6. Send a "your password was changed" notification email.

Common mistakes: predictable tokens, tokens that never expire, reusable tokens, leaking token in Referer headers (set `Referrer-Policy: no-referrer` on the reset page), using a host header to build the link.

Session cookies vs JWT: trade-offs?

MID

- **Server sessions:** random session id in an httpOnly cookie, data in Redis/DB. Instant revocation, simple; needs a shared store.
- **JWT:** signed token carrying claims; stateless verification, good across services. Hard to revoke before expiry, larger, easy to misuse.
- Common hybrid: short-lived JWT access token (5–15 min) + long-lived refresh token stored server-side and rotated.

What is inside a JWT and how is it verified?

MID

`header.payload.signature`, base64url-encoded. Header has `alg / kid`; payload has claims (`sub`, `exp`, `iat`, `iss`, `aud`, roles). The payload is **readable by anyone**; don't put secrets in it. Verify signature with the expected algorithm (reject `alg: none`), then check `exp`, `iss`, `aud`. HS256 uses a shared secret; RS256/ES256 use a private key to sign and a public key (JWKS) to verify.

Where should tokens be stored in the browser?

MID

Refresh token (or session id) in an `HttpOnly; Secure; SameSite=Lax/Strict` cookie so JS can't read it. Access token in memory. Avoid localStorage (XSS can read it). If using cookies, protect state-changing requests against CSRF (SameSite + CSRF token or checking `Origin`).

Explain refresh token rotation and reuse detection.

SENIOR

Each time a refresh token is used, issue a new one and invalidate the old one. All tokens from one login form a "family". If an old (already used) token shows up again, someone stole it: revoke the whole family and force re-login. Store refresh tokens hashed with device info and expiry.

Explain OAuth 2.0 Authorization Code flow with PKCE and OpenID Connect.

SENIOR

1. App creates `code_verifier` and its SHA-256 `code_challenge`.
2. Redirect user to the provider (Google, Entra ID) with `client_id`, `redirect_uri`, `scope`, `state`, `code_challenge`.
3. User logs in and consents; provider redirects back with a `code`.
4. App's backend exchanges code + `code_verifier` for tokens.
5. OIDC adds an **ID token** (who the user is); OAuth's access token is for calling APIs.

Follow-up: `state` prevents CSRF on the callback; PKCE stops intercepted codes from being used. Implicit flow is deprecated.

How do you protect login against brute force and credential stuffing?

SENIOR

- Rate limit per IP and per account (sliding window in Redis), progressive delays.
- Temporary lockout or CAPTCHA after repeated failures (careful: lockouts can be abused for DoS).
- Check passwords against breached lists (HaveIBeenPwned k-anonymity API).
- MFA, anomaly detection (new device/location email), generic error "Invalid email or password".

How would you add MFA and passkeys?

SENIOR

- **TOTP:** store an encrypted shared secret, show QR code, verify 6-digit codes with a small time window; give one-time recovery codes (stored hashed).
- **Passkeys / WebAuthn:** device creates a key pair; server stores the public key and verifies signed challenges. Phishing-resistant, no shared secret.
- SMS OTP is the weakest option (SIM swapping).

Design RBAC for a multi-tenant SaaS.

SENIOR

Tables: users, tenants, memberships (user, tenant, role), roles → permissions. Every query is scoped by `tenant_id` (or Postgres row-level security). Check permissions server-side in a single policy function, not in UI code. For complex rules consider ABAC or tools like OpenFGA/Cerbos.

Walk through the full login architecture end to end.

SENIOR

1. Next.js form posts to the API over HTTPS.
2. API rate-limits, loads user by email, verifies Argon2 hash, checks `email_verified` and MFA.
3. Issues short-lived access token + rotated refresh token (httpOnly cookie); records session (device, IP) in DB/Redis.
4. Next.js middleware checks the cookie for route protection; the API validates the token on every call.
5. Logout deletes the session/refresh token server-side and clears cookies.
6. Audit log for logins, resets and MFA changes.

See the diagrams at the top of this section.

How does logout work with JWTs if they're stateless?

SENIOR

Delete the refresh token server-side and clear cookies; the short-lived access token expires within minutes. For instant revocation keep a denylist of token `jti` s (in Redis, until expiry) or a per-user `token_version` that is checked on each request.

Azure

App Service, Functions, Entra ID, Key Vault, storage, DevOps.

IaaS vs PaaS vs SaaS with Azure examples?

JUNIOR

IaaS: Azure Virtual Machines (you manage OS). PaaS: App Service, Azure SQL, Functions (Azure manages OS and runtime). SaaS: Microsoft 365. Full stack apps usually run on PaaS.

What is Azure App Service?

JUNIOR

Managed hosting for web apps and APIs (Python, Node, .NET, containers). Provides autoscaling, deployment slots, custom domains, TLS, built-in auth ("Easy Auth"), and CI/CD integration.

What are resource groups, subscriptions and regions?

JUNIOR

A subscription is a billing and access boundary. Resource groups are logical containers for resources that share a lifecycle (delete the group, delete everything). Regions are the physical datacenter locations; pick close to users and pair for DR.

What are deployment slots and why use them?

MID

Separate live environments (e.g. `staging`) of the same App Service. Deploy to staging, warm it up, test, then **swap** with production for zero-downtime release and instant rollback by swapping back.

Azure Functions: triggers, bindings, and hosting plans?

MID

Serverless functions started by triggers (HTTP, Timer, Queue, Service Bus, Blob, Event Grid, Cosmos DB). Bindings connect inputs/outputs declaratively. Plans: Consumption / Flex Consumption (pay per execution, cold starts), Premium (pre-warmed, VNet), Dedicated (App Service plan). Durable Functions add stateful workflows.

What is Microsoft Entra ID (Azure AD) and how does it relate to app login?

MID

Azure's identity service: users, groups, app registrations, and OAuth 2.0 / OpenID Connect tokens. Use it for employee SSO and API protection. For customer-facing login (sign-up, social logins, password reset) use **Entra External ID** (successor to Azure AD B2C). In code, use MSAL (`msal-browser`, `msal` for Python) or Auth.js with the Entra provider.

What is a Managed Identity? System vs user-assigned?

MID

An identity in Entra ID that Azure manages for a resource, so code can access Key Vault, Storage or SQL **without any secret** (`DefaultAzureCredential`). System-assigned lives and dies with one resource; user-assigned is standalone and can be shared across resources.

How do you manage secrets on Azure?

MID

Store in Azure Key Vault; grant access with managed identity + RBAC; reference them in App Service settings as `@Microsoft.KeyVault(SecretUri=...)`. Never commit secrets; rotate them; enable soft-delete and purge protection.

Blob Storage tiers and SAS tokens?

MID

Tiers: Hot, Cool, Cold, Archive (cheaper storage, costlier access). A SAS (Shared Access Signature) grants time-limited, scoped access to a blob, e.g. let the browser upload a file directly without going through your API. Prefer user-delegation SAS (signed with Entra credentials).

Service Bus vs Storage Queues vs Event Grid vs Event Hubs?

MID

- **Storage Queue:** simple, cheap, large backlog.
- **Service Bus:** enterprise messaging: FIFO sessions, topics/subscriptions, dead-lettering, transactions.
- **Event Grid:** reactive event routing (blob created → function).
- **Event Hubs:** high-throughput streaming/telemetry.

How would you architect a Next.js + Python app on Azure?

SENIOR

- Front Door (CDN + WAF) → Next.js on App Service / Container Apps.
- FastAPI on Container Apps or App Service, autoscaled.
- Azure Database for PostgreSQL / Azure SQL; Azure Cache for Redis for sessions and rate limits.
- Blob Storage for uploads; Service Bus + Functions for emails and background jobs; Azure Communication Services or SendGrid for email.
- Entra External ID for customer auth; Key Vault + managed identity for secrets.
- Application Insights + Log Analytics for monitoring; private endpoints + VNet for DB.
- Bicep/Terraform for infra; GitHub Actions or Azure DevOps for CI/CD.

How do you set up CI/CD to Azure?

SENIOR

GitHub Actions or Azure Pipelines: lint + test → build container → push to Azure Container Registry → deploy to staging slot → smoke tests → swap to production. Authenticate with OIDC federated credentials (no stored secrets). Infra as code with Bicep or Terraform in the same pipeline.

How do you monitor and troubleshoot an Azure app?

SENIOR

Application Insights (requests, dependencies, exceptions, distributed tracing via OpenTelemetry), Log Analytics with KQL queries, alerts on failure rate and latency, availability tests, App Service diagnostics and log stream.

Azure RBAC vs Entra ID roles?

SENIOR

Azure RBAC controls access to Azure resources (Contributor on a resource group) with scope inheritance: management group → subscription → resource group → resource. Entra ID roles control directory administration (User Administrator). Grant least privilege at the narrowest scope.

System Design & DevOps

Scaling, caching, queues, CI/CD, observability.

How would you scale a web application?

MID

Stateless app servers behind a load balancer (scale horizontally), CDN for static assets, caching (Redis) for hot reads, database read replicas and indexing, queues for slow work, and later sharding or splitting services where there is a real bottleneck.

Caching strategies and invalidation?

MID

Cache-aside (app reads cache, falls back to DB, then fills cache), write-through, write-behind. Invalidate by TTL, by explicit delete on write, or by versioned keys. Browser/CDN caching via Cache-Control, ETag. Watch for cache stampedes (use locks or stale-while-revalidate).

Monolith vs microservices?

MID

Start with a well-structured modular monolith: simpler deploys, transactions and debugging. Split out services when teams or scaling needs differ; microservices add network failures, distributed data and ops overhead.

How do you implement rate limiting?

MID

Token bucket or sliding window counters in Redis, keyed by user/IP/API key. Return 429 with Retry-After. Apply at the gateway (Front Door, API Management) and in the app for sensitive endpoints like login and password reset.

Design a URL shortener / notification system / file upload service.

SENIOR

Use a repeatable structure:

1. Clarify requirements and scale (users, QPS, read/write ratio).
2. API design and data model.
3. High-level diagram: client, CDN, LB, services, DB, cache, queue.
4. Deep dive into the hardest part (ID generation, fan-out, large uploads via presigned/SAS URLs).
5. Bottlenecks, failure modes, monitoring, trade-offs.

What is the CAP theorem?

SENIOR

During a network partition a distributed store must choose Consistency (reject or block) or Availability (serve possibly stale data). Without partitions you trade latency vs consistency (PACELC). Cosmos DB exposes five consistency levels for this reason.

How do you make a system observable?

SENIOR

Structured logs with a correlation/request id, metrics (RED: rate, errors, duration), distributed traces with OpenTelemetry, dashboards and SLO-based alerts. Log auth events without logging passwords or tokens.

What is idempotency and why does it matter for APIs and queues?

SENIOR

Repeating an operation has the same effect as doing it once. Queues deliver at-least-once and clients retry on timeouts, so handlers must dedupe (idempotency key stored with the result, unique constraints, upserts).

What happens when you type a URL and press Enter?

JUNIOR

DNS lookup → TCP + TLS handshake → HTTP request → server/CDN responds → browser parses HTML, builds DOM and CSSOM, runs JS, lays out, paints. Mention caching, HTTP/2 or HTTP/3, and where SSR fits.

Docker basics: image vs container, multi-stage builds?

MID

An image is a read-only template; a container is a running instance. Multi-stage builds compile in one stage and copy only the output into a slim runtime image (e.g. Next.js `standalone` output on `node:alpine`, FastAPI on `python:slim`).

Web Security

XSS, CSRF, CORS, SQL injection, headers.

What is XSS and how do you prevent it?

JUNIOR

Cross-Site Scripting: attacker-controlled script runs in your page. React escapes text by default; danger comes from `dangerouslySetInnerHTML`, `href="javascript:..."`, and unsanitized HTML. Prevent with output encoding, DOMPurify for user HTML, a Content-Security-Policy, and `httpOnly` cookies.

What is CSRF and how do you prevent it?

MID

Cross-Site Request Forgery: another site makes the victim's browser send an authenticated request with their cookies. Prevent with `SameSite=Lax/Strict` cookies, CSRF tokens, checking `Origin`, and never changing state on GET. Next.js Server Actions check Origin automatically.

What is CORS?

MID

A browser rule that blocks JS from reading responses from another origin unless the server allows it with `Access-Control-Allow-Origin`. Non-simple requests send an OPTIONS preflight. CORS is not a security wall for your API (curl ignores it); it protects users' browsers. Never use `*` with credentials.

How do you prevent SQL injection?

MID

Parameterized queries or an ORM; never build SQL with string formatting. Validate input, least-privilege DB user.

```
cur.execute("SELECT * FROM users WHERE email = %s", (email,))
```

What are the OWASP Top 10?

MID

Broken access control, cryptographic failures, injection, insecure design, security misconfiguration, vulnerable components, identification and authentication failures, software and data integrity failures, logging and monitoring failures, SSRF. Broken access control is number one: always check ownership on the server (`WHERE id=:id AND user_id=:me`).

Which security headers do you set?

SENIOR

`Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `X-Frame-Options / frame-ancestors`, `Permissions-Policy`. Set in `next.config.js` headers or middleware.

What is SSRF?

SENIOR

Server-Side Request Forgery: the attacker makes your server fetch a URL they choose, e.g. an internal service or cloud metadata endpoint (`169.254.169.254`). Prevent by allow-listing domains, blocking private IP ranges after DNS resolution, and isolating fetchers.

DSA: Strings, Arrays, Lists

The data structures and algorithms problems asked most often in full stack rounds, with Python and JavaScript solutions and complexity.

Reverse a string and check if it is a palindrome (ignore case and non-letters).

JUNIOR

Approach: two pointers moving inward, skipping non-alphanumerics. **Time** $O(n)$, **space** $O(1)$.

```
# Python
def is_palindrome(s: str) -> bool:
    i, j = 0, len(s) - 1
    while i < j:
        while i < j and not s[i].isalnum(): i += 1
        while i < j and not s[j].isalnum(): j -= 1
        if s[i].lower() != s[j].lower(): return False
        i, j = i + 1, j - 1
    return True
# reverse: s[::-1]

// JavaScript
const reverse = s => [...s].reverse().join('');
function isPalindrome(s) {
    const t = s.toLowerCase().replace(/^[a-z0-9]/g, '');
    let i = 0, j = t.length - 1;
    while (i < j) if (t[i++] !== t[j--]) return false;
    return true;
}
```

Valid anagram: are two strings anagrams?

JUNIOR

Approach: count characters. $O(n)$ time, $O(k)$ space for alphabet size k .

```
# Python
from collections import Counter
def is_anagram(a, b): return Counter(a) == Counter(b)

// JavaScript
function isAnagram(a, b) {
    if (a.length !== b.length) return false;
    const m = {};
    for (const c of a) m[c] = (m[c] || 0) + 1;
    for (const c of b) { if (!m[c]) return false; m[c]--; }
    return true;
}
```

First non-repeating character in a string.

JUNIOR

Approach: one pass to count, second pass to find first count of 1. $O(n)$.

```
# Python
from collections import Counter
def first_unique(s):
    c = Counter(s)
    return next((ch for ch in s if c[ch] == 1), None)

// JavaScript
function firstUnique(s) {
    const m = new Map();
    for (const c of s) m.set(c, (m.get(c) || 0) + 1);
    for (const c of s) if (m.get(c) === 1) return c;
    return null;
}
```

Count character / word frequency; find the most frequent character.

JUNIOR

```
# Python
from collections import Counter
Counter("hello world".split())          # word counts
Counter("banana").most_common(1)        # [('a', 3)]

// JavaScript
function mostFrequent(s) {
    const m = {}; let best = '', max = 0;
    for (const c of s) {
        m[c] = (m[c] || 0) + 1;
        if (m[c] > max) { max = m[c]; best = c; }
    }
    return best;
}
```

Two Sum: return indices of two numbers adding to a target.

JUNIOR

Approach: hash map of value $\rightarrow$ index; for each number check if `target - x` was seen. $O(n)$ time, $O(n)$ space (brute force is $O(n^2)$).

```
# Python
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen: return [seen[target - x], i]
        seen[x] = i

// JavaScript
function twoSum(nums, target) {
    const seen = new Map();
    for (let i = 0; i < nums.length; i++) {
        const need = target - nums[i];
        if (seen.has(need)) return [seen.get(need), i];
        seen.set(nums[i], i);
    }
}
```

Remove duplicates from an array (keep order); remove duplicates in place from a sorted array.

JUNIOR

```
# Python
list(dict.fromkeys(arr))          # keeps order

def dedupe_sorted(nums):          # in place, returns new length
    k = 0
    for x in nums:
        if k == 0 or x != nums[k - 1]:
            nums[k] = x; k += 1
    return k

// JavaScript
const unique = arr => [...new Set(arr)];
```

Find the largest and second largest numbers in one pass.

JUNIOR

```
# Python
def top_two(nums):
    a = b = float('-inf')
    for x in nums:
        if x > a: a, b = x, a
        elif a > x > b: b = x
    return a, b

// JavaScript
function topTwo(nums) {
    let a = -Infinity, b = -Infinity;
    for (const x of nums) {
        if (x > a) { b = a; a = x; }
        else if (x > b && x !== a) b = x;
    }
    return [a, b];
}
```

Move all zeroes to the end, keeping order of others.

JUNIOR

Two pointers, in place. $O(n)$ time, $O(1)$ space.

```
# Python
def move_zeroes(nums):
    k = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[k], nums[i] = nums[i], nums[k]; k += 1

// JavaScript
function moveZeroes(nums) {
    let k = 0;
    for (let i = 0; i < nums.length; i++)
        if (nums[i] !== 0) { [nums[k], nums[i]] = [nums[i], nums[k]]; k++; }
}
```

Longest substring without repeating characters.

MID

Approach: sliding window with last-seen index map. $O(n)$ time.

```
# Python
def longest_unique(s):
    last, start, best = {}, 0, 0
    for i, c in enumerate(s):
        if c in last and last[c] >= start: start = last[c] + 1
        last[c] = i
        best = max(best, i - start + 1)
    return best

// JavaScript
function longestUnique(s) {
    const last = new Map(); let start = 0, best = 0;
    for (let i = 0; i < s.length; i++) {
        if (last.has(s[i]) && last.get(s[i]) >= start) start = last.get(s[i]) + 1;
        last.set(s[i], i);
        best = Math.max(best, i - start + 1);
    }
    return best;
}
// "abcabcbb" -> 3
```

Valid parentheses: `()[]{}` balanced?

MID

Approach: stack. Push openers; on closer, top must match. $O(n)$.

```
# Python
def valid(s):
    pairs, st = {')': '(', ']': '[', '}': '{'}, []
    for c in s:
        if c in pairs:
            if not st or st.pop() != pairs[c]: return False
        else: st.append(c)
    return not st

// JavaScript
function valid(s) {
    const pairs = { ')': '(', ']': '[', '}': '{' }, st = [];
    for (const c of s) {
        if (pairs[c]) { if (st.pop() !== pairs[c]) return false; }
        else st.push(c);
    }
    return st.length === 0;
}
```

Group anagrams together.

MID

Approach: key = sorted word (or 26-letter count tuple). $O(n \cdot k \log k)$.

```
# Python
from collections import defaultdict
def group_anagrams(words):
    g = defaultdict(list)
    for w in words: g[''.join(sorted(w))].append(w)
    return list(g.values())

// JavaScript
function groupAnagrams(words) {
    const g = new Map();
    for (const w of words) {
        const k = [...w].sort().join('');
        g.set(k, [...(g.get(k) || []), w]);
    }
    return [...g.values()];
}
```

Longest common prefix of an array of strings.

MID

```
# Python
def lcp(strs):
    if not strs: return ""
    for i, ch in enumerate(strs[0]):
        if any(i >= len(s) or s[i] != ch for s in strs[1:]):
            return strs[0][:i]
    return strs[0]

// JavaScript
function lcp(strs) {
    if (!strs.length) return '';
    let p = strs[0];
    for (const s of strs) while (!s.startsWith(p)) p = p.slice(0, -1);
    return p;
}
```

String compression: `aaabbc` → `a3b2c1` .

MID

```
# Python
from itertools import groupby
def compress(s):
    out = ''.join(f"{k}{len(list(g))}" for k, g in groupby(s))
    return out if len(out) < len(s) else s

// JavaScript
function compress(s) {
    let out = '';
    for (let i = 0; i < s.length; ) {
        let j = i;
        while (j < s.length && s[j] === s[i]) j++;
        out += s[i] + (j - i);
        i = j;
    }
    return out.length < s.length ? out : s;
}
```

Maximum subarray sum (Kadane's algorithm).

MID

At each index, either extend the current subarray or start fresh. $O(n)$, $O(1)$.

```
# Python
def max_subarray(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

// JavaScript
function maxSubarray(nums) {
    let cur = nums[0], best = nums[0];
    for (let i = 1; i < nums.length; i++) {
        cur = Math.max(nums[i], cur + nums[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

Best time to buy and sell stock (one transaction).

MID

```
# Python
def max_profit(prices):
    lo, best = float('inf'), 0
    for p in prices:
        lo = min(lo, p)
        best = max(best, p - lo)
    return best

// JavaScript
function maxProfit(prices) {
    let lo = Infinity, best = 0;
    for (const p of prices) { lo = Math.min(lo, p); best = Math.max(best, p - lo); }
    return best;
}
```

Product of array except self (no division).

MID

Prefix products left to right, then multiply by suffix products right to left. $O(n)$, $O(1)$ extra.

```
# Python
def product_except_self(nums):
    n = len(nums); out = [1] * n
    left = 1
    for i in range(n): out[i] = left; left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1): out[i] *= right; right *= nums[i]
    return out

// JavaScript
function productExceptSelf(nums) {
    const out = Array(nums.length).fill(1);
    for (let i = 0, l = 1; i < nums.length; i++) { out[i] = l; l *= nums[i]; }
    for (let i = nums.length - 1, r = 1; i >= 0; i--) { out[i] *= r; r *= nums[i]; }
    return out;
}
```

Rotate an array right by k steps in place.

MID

Reverse all, reverse first k, reverse the rest. $O(n)$, $O(1)$.

```
# Python
def rotate(nums, k):
    k %= len(nums)
    def rev(i, j):
        while i < j: nums[i], nums[j] = nums[j], nums[i]; i += 1; j -= 1
    rev(0, len(nums) - 1); rev(0, k - 1); rev(k, len(nums) - 1)

// JavaScript
function rotate(nums, k) {
    k %= nums.length;
    const rev = (i, j) => {
        for (; i < j; i++, j--) [nums[i], nums[j]] = [nums[j], nums[i]];
    };
    rev(0, nums.length - 1); rev(0, k - 1); rev(k, nums.length - 1);
}
```

Merge overlapping intervals.

MID

Sort by start, then extend or push. $O(n \log n)$.

```
# Python
def merge(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]: out[-1][1] = max(out[-1][1], e)
        else: out.append([s, e])
    return out

// JavaScript
function merge(iv) {
    iv.sort((a, b) => a[0] - b[0]);
    const out = [];
    for (const [s, e] of iv) {
        const last = out[out.length - 1];
        if (last && s <= last[1]) last[1] = Math.max(last[1], e);
        else out.push([s, e]);
    }
    return out;
}
```

Merge two sorted arrays into one sorted array.

MID

```
# Python
def merge_sorted(a, b):
    i = j = 0; out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]: out.append(a[i]); i += 1
        else: out.append(b[j]); j += 1
    return out + a[i:] + b[j:]

// JavaScript
function mergeSorted(a, b) {
    const out = []; let i = 0, j = 0;
    while (i < a.length && j < b.length) out.push(a[i] <= b[j] ? a[i++] : b[j++]);
    return out.concat(a.slice(i), b.slice(j));
}
```

Top K frequent elements.

MID

Count, then bucket sort by frequency for $O(n)$, or use a heap for $O(n \log k)$.

```
# Python
from collections import Counter
import heapq
def top_k(nums, k):
    return [x for x, _ in Counter(nums).most_common(k)]
    # or: heapq.nlargest(k, cnt.keys(), key=cnt.get)

// JavaScript
function topK(nums, k) {
    const cnt = new Map();
    for (const x of nums) cnt.set(x, (cnt.get(x) || 0) + 1);
    const buckets = Array(nums.length + 1).fill().map(() => []);
    for (const [x, c] of cnt) buckets[c].push(x);
    return buckets.flat().reverse().slice(0, k);
}
```

3Sum: all unique triplets that sum to zero.

MID

Sort, fix one number, two-pointer on the rest, skip duplicates. $O(n^2)$.

```
# Python
def three_sum(nums):
    nums.sort(); res = []
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i - 1]: continue
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l + 1]: l += 1
                l += 1; r -= 1
    return res
```

Binary search, and find first/last position of a target.

MID

```
# Python
def lower_bound(a, t):          # first index with a[i] >= t
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < t: lo = mid + 1
        else: hi = mid
    return lo
# first = lower_bound(a, t); last = lower_bound(a, t + 1) - 1

// JavaScript
function binarySearch(a, t) {
    let lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        const mid = (lo + hi) >> 1;
        if (a[mid] === t) return mid;
        a[mid] < t ? lo = mid + 1 : hi = mid - 1;
    }
    return -1;
}
```

$O(\log n)$. Only works on sorted input.

Reverse a singly linked list (iterative and recursive).

MID

```
# Python
class Node:
    def __init__(self, val, next=None): self.val, self.next = val, next

def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

def reverse_rec(head):
    if not head or not head.next: return head
    new_head = reverse_rec(head.next)
    head.next.next = head; head.next = None
    return new_head

// JavaScript
function reverse(head) {
    let prev = null;
    while (head) { const nxt = head.next; head.next = prev; prev = head; head = nxt; }
    return prev;
}
```

Detect a cycle in a linked list; find the middle node.

MID

Floyd's fast/slow pointers. $O(n)$, $O(1)$.

```
# Python
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast: return True
    return False

def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow
```

Merge two sorted linked lists.

MID

```
# Python
def merge_lists(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.val <= b.val: tail.next, a = a, a.next
        else: tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

Design an LRU cache with O(1) get and put.

SENIOR

Hash map + doubly linked list (most recent at the end). In Python, `OrderedDict` gives this; in JS, a `Map` keeps insertion order.

```
# Python
from collections import OrderedDict
class LRU:
    def __init__(self, cap): self.cap, self.d = cap, OrderedDict()
    def get(self, k):
        if k not in self.d: return -1
        self.d.move_to_end(k); return self.d[k]
    def put(self, k, v):
        self.d[k] = v; self.d.move_to_end(k)
        if len(self.d) > self.cap: self.d.popitem(last=False)

// JavaScript
class LRU {
    constructor(cap) { this.cap = cap; this.m = new Map(); }
    get(k) {
        if (!this.m.has(k)) return -1;
        const v = this.m.get(k); this.m.delete(k); this.m.set(k, v); return v;
    }
    put(k, v) {
        this.m.delete(k); this.m.set(k, v);
        if (this.m.size > this.cap) this.m.delete(this.m.keys().next().value);
    }
}
```

Implement a stack and a queue; a queue using two stacks.

MID

```
# Python
from collections import deque
stack = []; stack.append(1); stack.pop()
q = deque(); q.append(1); q.popleft()           # 0(1) both ends

class QueueTwoStacks:
    def __init__(self): self.inp, self.out = [], []
    def push(self, x): self.inp.append(x)
    def pop(self):
        if not self.out:
            while self.inp: self.out.append(self.inp.pop())
        return self.out.pop()                 # amortized 0(1)
```

Follow-up: `list.pop(0)` and JS `array.shift()` are $O(n)$; use deque or an index pointer.

Binary tree: max depth, level order (BFS), and inorder traversal.

MID

```
# Python
from collections import deque
def max_depth(root):
    return 0 if not root else 1 + max(max_depth(root.left), max_depth(root.right))

def level_order(root):
    if not root: return []
    res, q = [], deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            n = q.popleft(); level.append(n.val)
            if n.left: q.append(n.left)
            if n.right: q.append(n.right)
        res.append(level)
    return res

def inorder(root):
    return inorder(root.left) + [root.val] + inorder(root.right) if root else []
```

Validate a binary search tree.

MID

```
# Python
def is_bst(node, lo=float('-inf'), hi=float('inf')):
    if not node: return True
    if not lo < node.val < hi: return False
    return is_bst(node.left, lo, node.val) and is_bst(node.right, node.val, hi)
```

Common mistake: only comparing a node with its direct children.

Number of islands (grid DFS/BFS).

MID

```
# Python
def num_islands(grid):
    R, C = len(grid), len(grid[0]); count = 0
    def sink(r, c):
        if 0 <= r < R and 0 <= c < C and grid[r][c] == '1':
            grid[r][c] = '0'
            sink(r+1, c); sink(r-1, c); sink(r, c+1); sink(r, c-1)
    for r in range(R):
        for c in range(C):
            if grid[r][c] == '1': count += 1; sink(r, c)
    return count
```

$O(R \cdot C)$. For huge grids use an explicit stack to avoid recursion limits.

Climbing stairs / Fibonacci with memoization and bottom-up DP.

MID

```
# Python
from functools import cache
@cache
def fib(n): return n if n < 2 else fib(n - 1) + fib(n - 2)

def climb(n):
    # ways using 1 or 2 steps
    a, b = 1, 1
    for _ in range(n): a, b = b, a + b
    return a
```

Naive recursion is $O(2^n)$; memoized or iterative is $O(n)$.

Coin change: minimum coins to make an amount.

SENIOR

```
# Python
def coin_change(coins, amount):
    dp = [0] + [float('inf')] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

$O(\text{amount} \times \text{coins})$. Greedy fails for coins like [1, 3, 4] and amount 6.

Kth largest element in an array.

SENIOR

Min-heap of size k: $O(n \log k)$. Quickselect: average $O(n)$.

```
# Python
import heapq
def kth_largest(nums, k):
    h = nums[:k]; heapq.heapify(h)
    for x in nums[k:]:
        if x > h[0]: heapq.heapreplace(h, x)
    return h[0]
```

Subarray sum equals K (count).

SENIOR

Prefix sums in a hash map: if `prefix - k` was seen, those subarrays end here. $O(n)$.

```
# Python
def subarray_sum(nums, k):
    seen, s, count = {0: 1}, 0, 0
    for x in nums:
        s += x
        count += seen.get(s - k, 0)
        seen[s] = seen.get(s, 0) + 1
    return count
```

Big-O cheat sheet: what should you know?

JUNIOR

- $O(1)$: hash lookup, array index, stack push/pop.
- $O(\log n)$: binary search, heap push/pop, balanced BST.
- $O(n)$: single loop, two pointers, sliding window.
- $O(n \log n)$: good sorts (Python's Timsort, JS's sort).
- $O(n^2)$: nested loops; say how you'd improve it.

Interview habit: state brute force first, then optimise, then give time and space complexity and test edge cases (empty, one element, duplicates, negatives).

Behavioral & Scenario

Project stories and debugging scenarios. Answer with Situation, Task, Action, Result.

Walk me through a project you built end to end.

JUNIOR

Structure: the problem and users → your stack and why (e.g. Next.js + FastAPI + PostgreSQL on Azure) → one hard technical decision → what you'd do differently → measurable result. Keep it to 2–3 minutes.

Tell me about a production bug you debugged.

MID

Use STAR. Show method: reproduce, read logs/traces (App Insights), form a hypothesis, isolate, fix, add a test and monitoring so it can't recur. Mention communication with the team during the incident.

An API endpoint became slow. How do you investigate?

MID

1. Confirm with metrics: which endpoint, since when, p95/p99.
2. Check recent deploys and traffic changes.
3. Trace the request: DB time vs external calls vs app CPU.
4. Run **EXPLAIN ANALYZE** on slow queries; look for missing indexes or $N+1$.
5. Fix, add caching if appropriate, and set an alert.

How do you handle disagreement on a technical decision?

MID

Understand their reasons, compare options against agreed criteria (cost, risk, time), prototype or measure when possible, decide and document (ADR), then commit fully even if your option lost.

How do you use AI coding tools and make sure the code is correct?

SENIOR

Use them for scaffolding, tests and exploring APIs; review every line, run tests and type checks, check security-sensitive code (auth, SQL, secrets) by hand, and never paste secrets or customer data into prompts. Interviewers in 2026 often ask this directly.

How do you estimate and break down a feature?

SENIOR

Split into thin vertical slices (UI + API + DB) that each ship value, identify unknowns and spike them first, estimate in ranges, and flag risks early. Example: "password reset" → token table + endpoint, email job, reset page, session revocation, rate limiting, tests.

Why should we hire you for a 4-year full stack role? What is your ownership story?

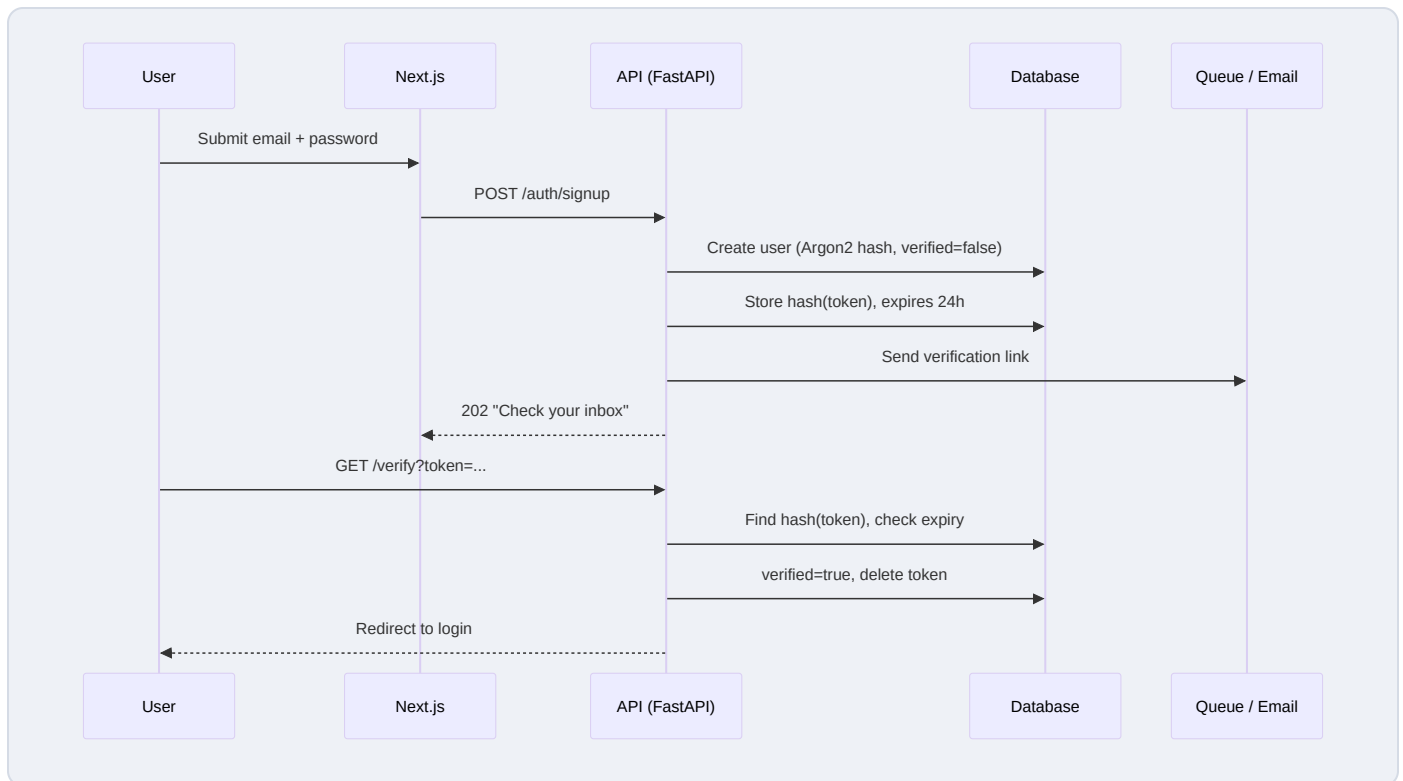
SENIOR

Show scope growth: a feature you owned from design to production, a decision you made (and its trade-off), how you mentored someone or improved a team process (code review, CI, testing), and a metric that moved (latency, conversion, incidents).

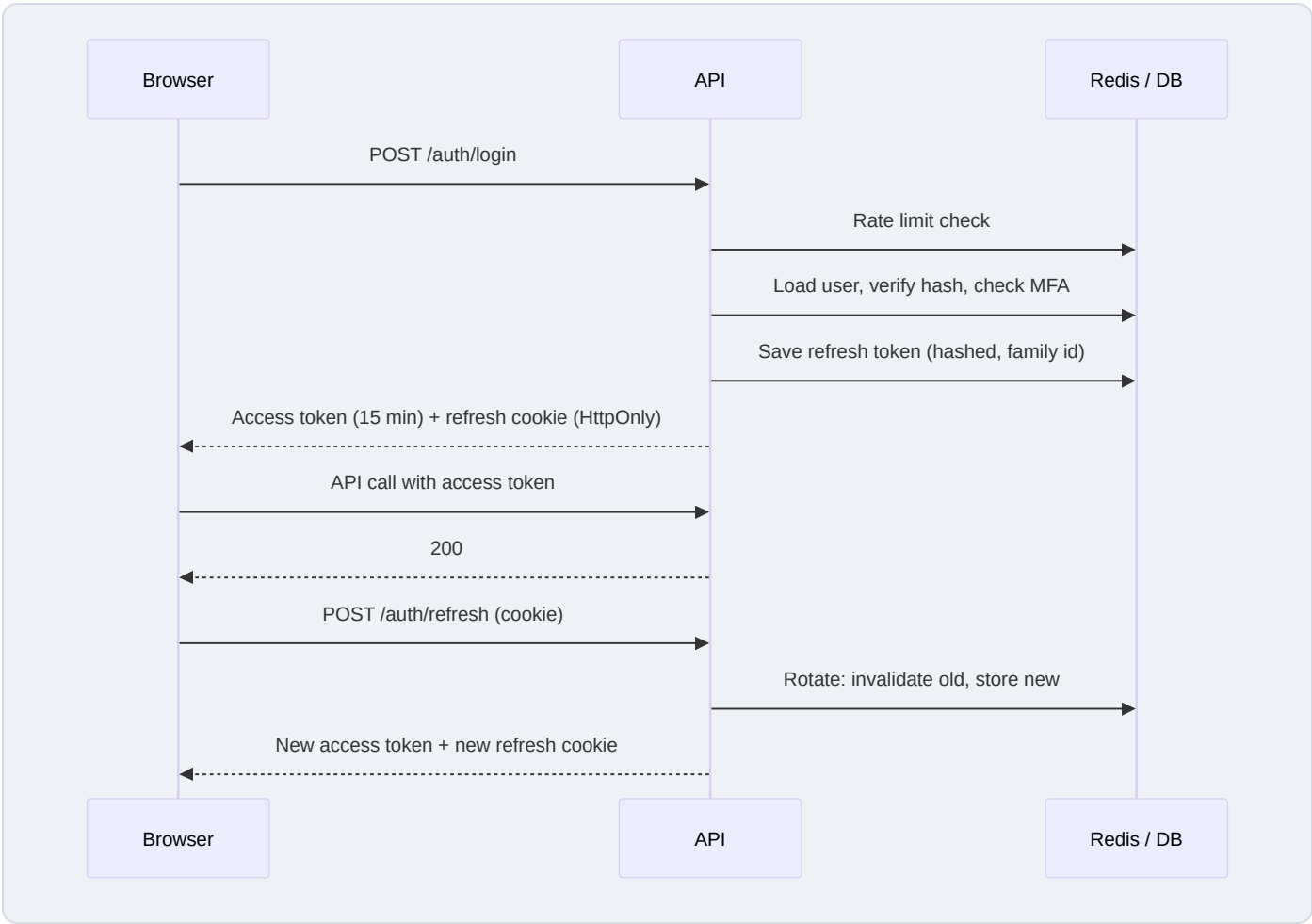
Architecture diagrams

Interviewers often ask you to draw these on a whiteboard. Practise explaining each arrow.

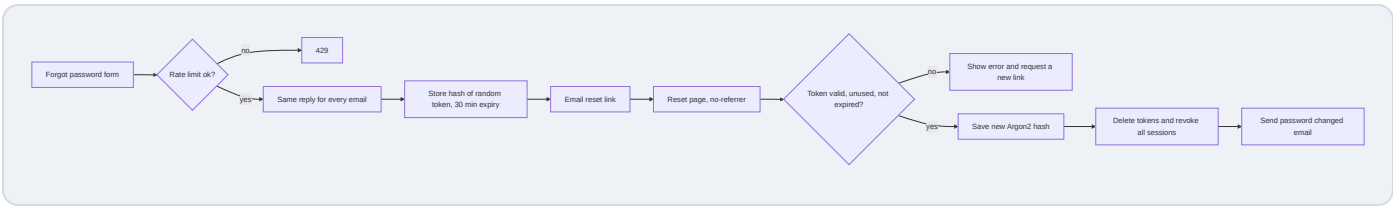
Signup and email verification



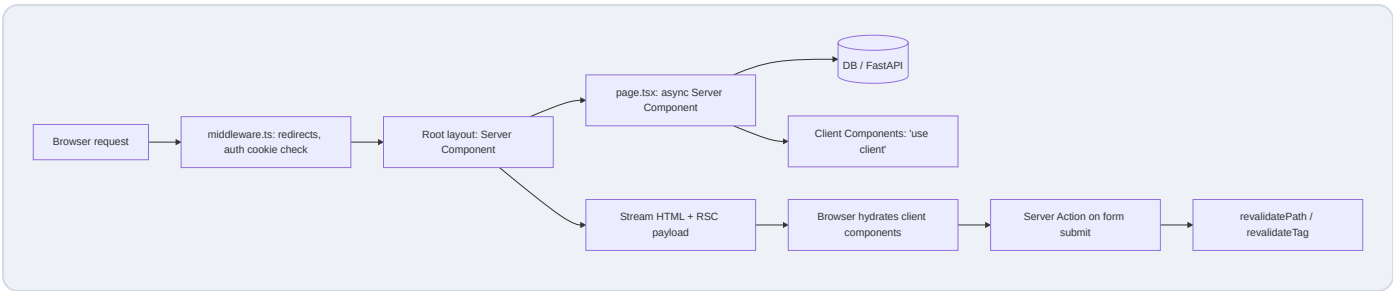
Login with access and refresh tokens



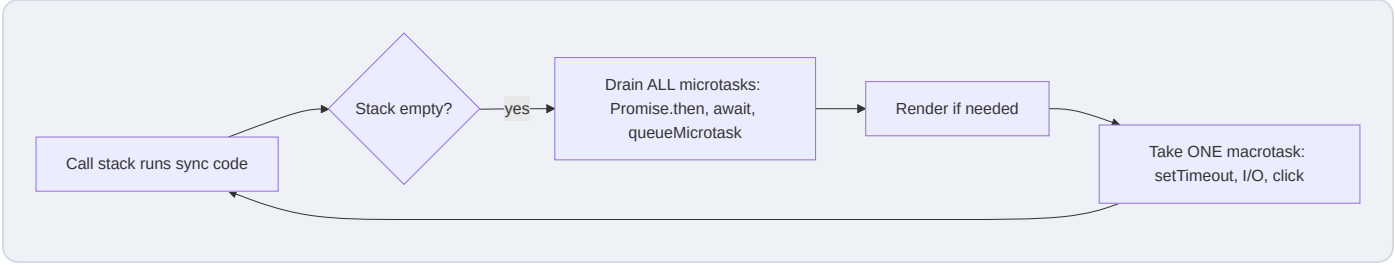
Forgot and reset password



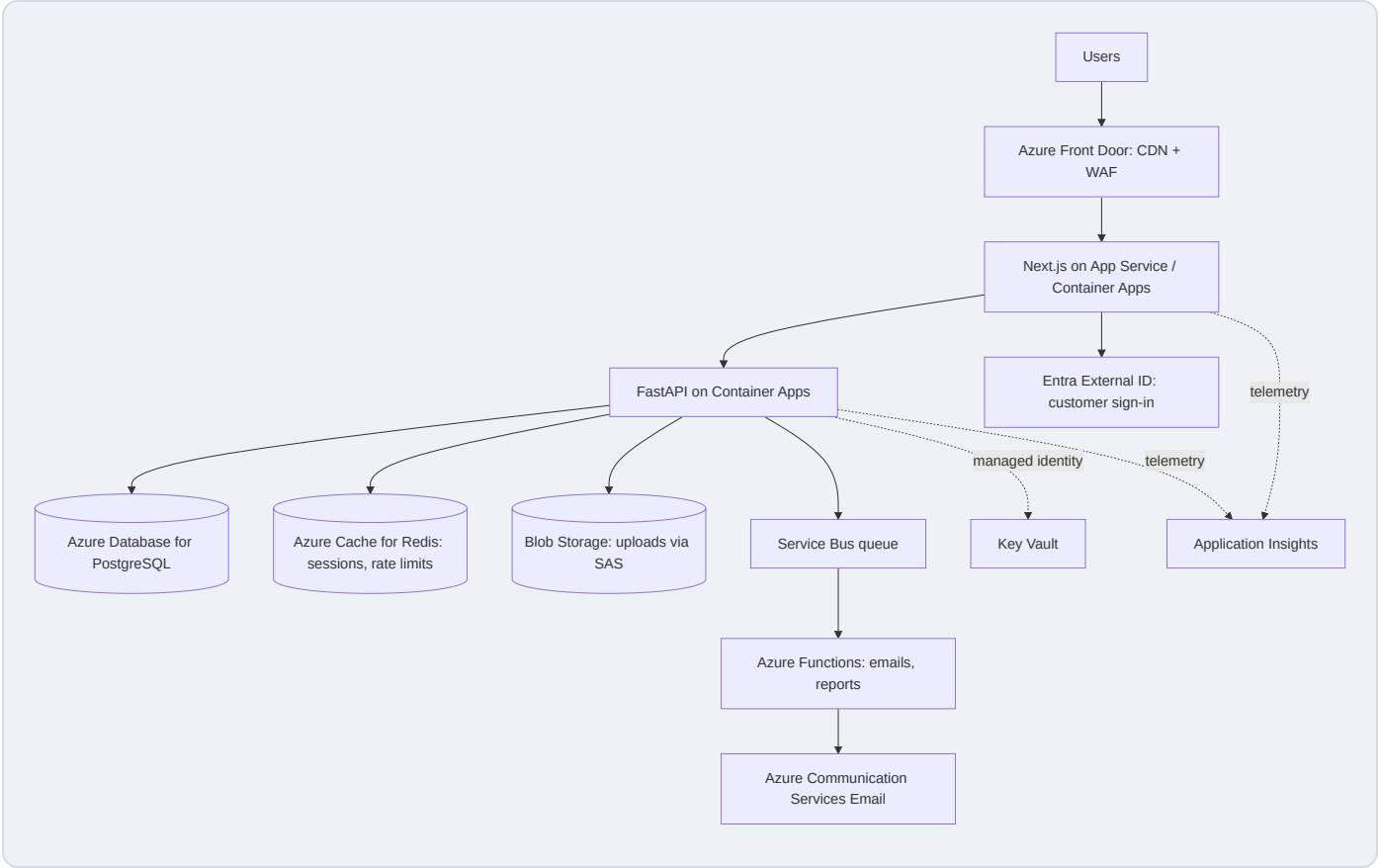
Next.js App Router request flow



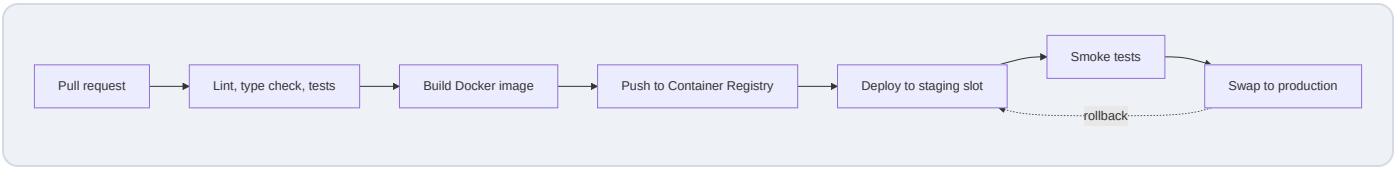
JavaScript event loop



Next.js + Python on Azure



CI/CD to Azure



Sources

Questions were cross-checked against these published guides (September 2026). Answers are written fresh and kept short; go deeper on anything you get asked twice.

- [Interview Kickstart: Full Stack Q&A 2026](#)
- [KORE1: Full Stack Developer Interview Questions 2026](#)
- [Nucamp: Top 30 Full Stack Questions 2026](#)
- [Edureka: 40+ Full Stack Questions](#)
- [TestMu: 100+ Full Stack Questions](#)
- [GreatFrontEnd: Next.js Questions \(Mid–Senior\)](#)
- [GreatFrontEnd: Next.js for Freshers](#)
- [InterviewBit: Next.js Questions](#)
- [DataCamp: 33 Azure Questions](#)
- [KodeKloud: Azure Questions 2026](#)
- [FullStack.Cafe: Azure Developer Questions](#)
- [Tenant365Admin: Microsoft Entra Questions](#)
- [GeeksforGeeks: Designing an Authentication System](#)
- [Chirag Hasija: Design a Login System](#)
- [freeCodeCamp: JWT and Refresh Tokens](#)
- [Interview Coder: 35 JWT Questions](#)
- [DEV: Sessions vs JWT vs Passkeys](#)